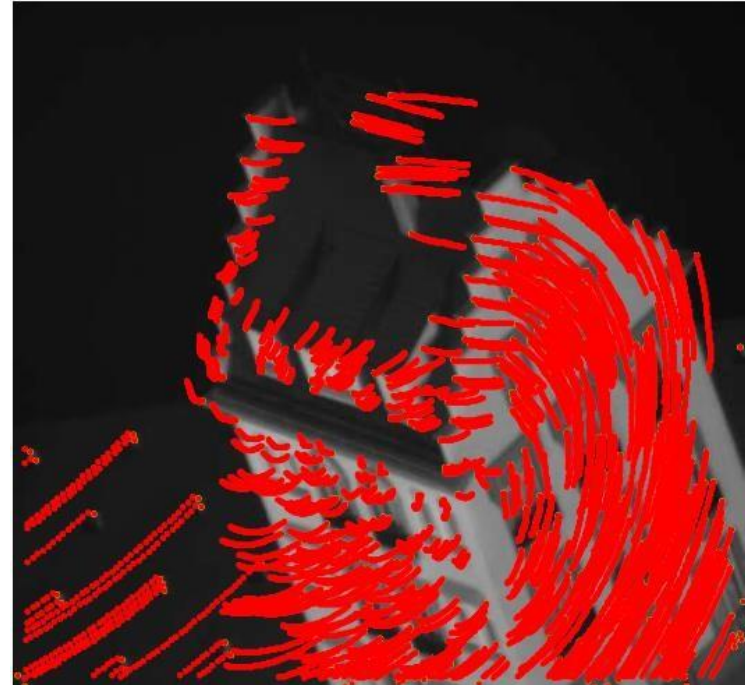
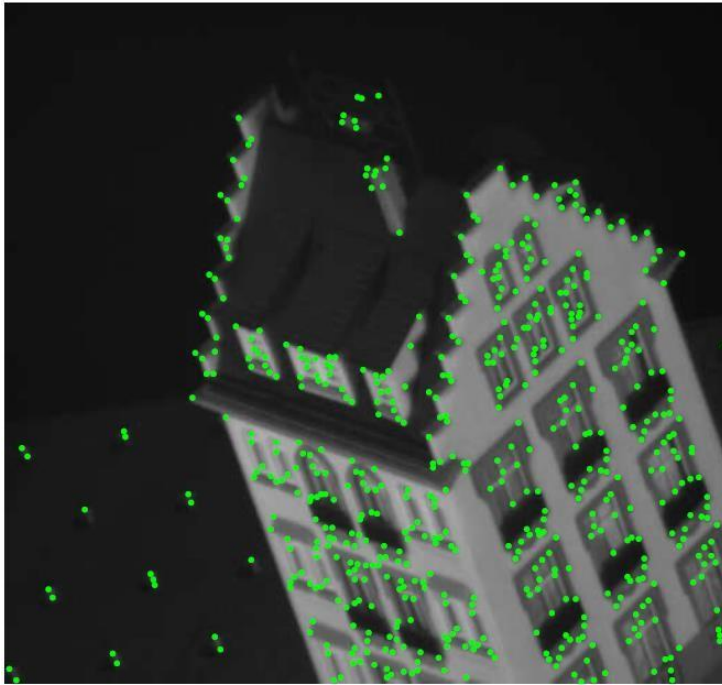


Feature Tracking and Optical Flow



Computer Vision

Adduru U G Sankararao, IIIT Sri City

This class: recovering motion

- Feature tracking
 - Extract visual features (corners, blobs, textured areas) and “track” them over multiple frames
- Optical flow
 - Definition: optical flow is the *apparent* motion of brightness patterns in the image
 - Recover image motion at each pixel from spatio-temporal image brightness variations

B. Lucas and T. Kanade. [An iterative image registration technique with an application to stereo vision](#). In *Proceedings of the International Joint Conference on Artificial Intelligence*, 1981.

Feature tracking - Challenges

- Figure out which features can be tracked

Feature tracking - Challenges

- Figure out which features can be tracked
- Efficiently track across frames

Feature tracking - Challenges

- Figure out which features can be tracked
- Efficiently track across frames
- Some points may change appearance over time (e.g., due to rotation, moving into shadows, etc.)

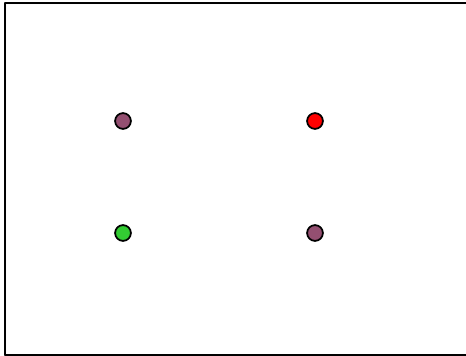
Feature tracking - Challenges

- Figure out which features can be tracked
- Efficiently track across frames
- Some points may change appearance over time (e.g., due to rotation, moving into shadows, etc.)
- Points may appear or disappear: need to be able to add/delete tracked points

Feature tracking - Challenges

- Figure out which features can be tracked
- Efficiently track across frames
- Some points may change appearance over time (e.g., due to rotation, moving into shadows, etc.)
- Points may appear or disappear: need to be able to add/delete tracked points
- Drift: small errors can accumulate as appearance model is updated

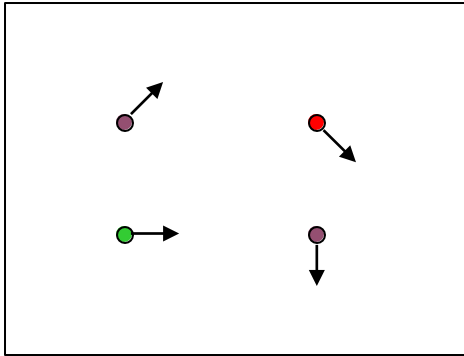
Feature tracking: Lucas-Kanade



$$I(x,y,t)$$

- Given two subsequent frames, estimate the point translation

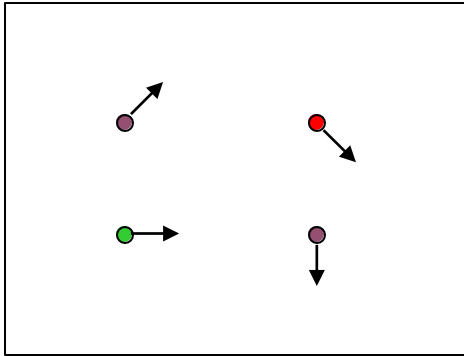
Feature tracking: Lucas-Kanade



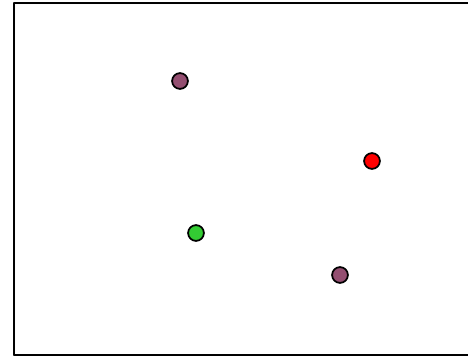
$$I(x,y,t)$$

- Given two subsequent frames, estimate the point translation

Feature tracking: Lucas-Kanade



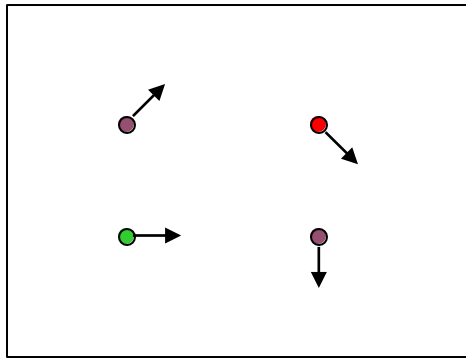
$I(x,y,t)$



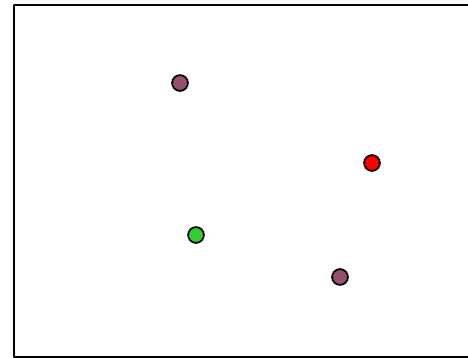
$I(x,y,t+1)$

- Given two subsequent frames, estimate the point translation

Feature tracking: Lucas-Kanade



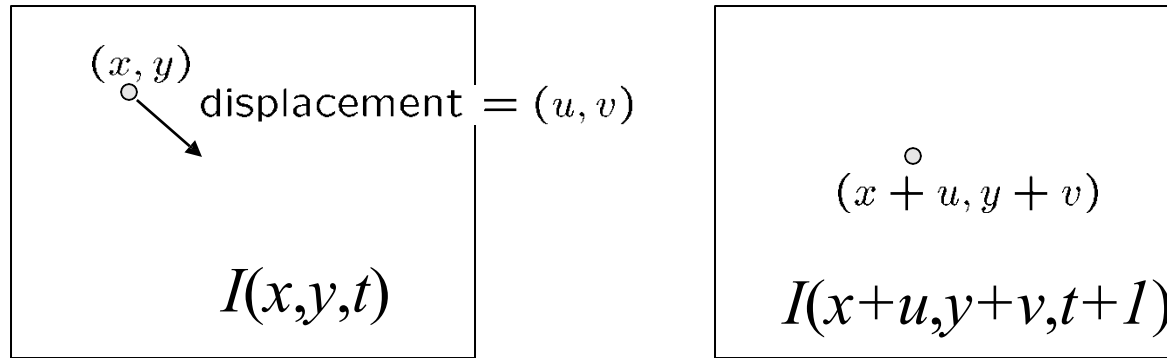
$I(x,y,t)$



$I(x,y,t+1)$

- Given two subsequent frames, estimate the point translation
- Key assumptions of Lucas-Kanade Tracker
 - **Brightness constancy:** Brightness of the same point looks the same in every frame
 - **Small motion:** points do not move very far (displacement and time step are small)
 - **Spatial coherence:** points move like their neighbors

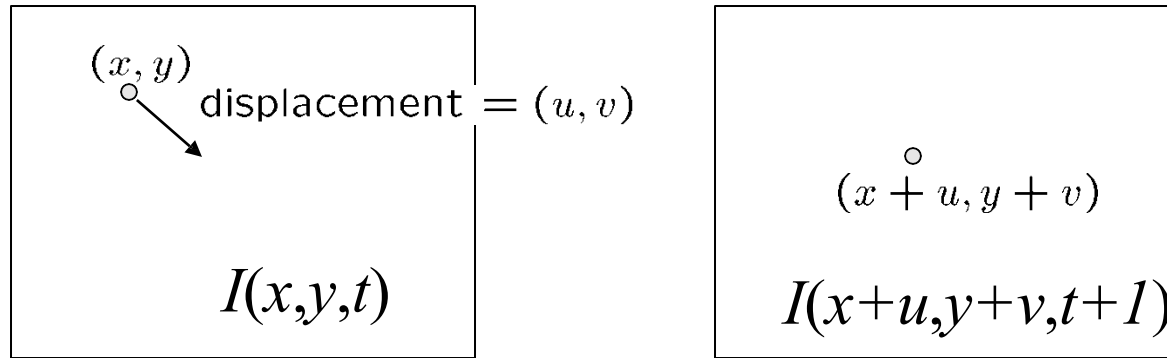
The brightness constancy constraint



- Brightness Constancy Equation:

$$I(x, y, t) = I(x + u, y + v, t + 1)$$

The brightness constancy constraint



- Brightness Constancy Equation:

$$I(x, y, t) = I(x + u, y + v, t + 1)$$

Take Taylor expansion of $I(x + u, y + v, t + 1)$ at (x, y, t) to linearize the right side:

$$I(x + u, y + v, t + 1) \approx I(x, y, t) + I_x \cdot u + I_y \cdot v + I_t$$

Taylor Series Expansion

Expand a function as an infinite sum of its derivatives

$$f(x + \delta x) = f(x) + \frac{\partial f}{\partial x} \delta x + \frac{\partial^2 f}{\partial x^2} \frac{\delta x^2}{2!} + \cdots + \frac{\partial^n f}{\partial x^n} \frac{\delta x^n}{n!} \quad \boxed{1}$$

If δx is small:

$$f(x + \delta x) = f(x) + \frac{\partial f}{\partial x} \delta x + \boxed{O(\delta x^2)} \rightarrow \text{Almost Zero}$$

For a function of three variables with small $\delta x, \delta y, \delta t$:

$$f(x + \delta x, y + \delta y, t + \delta t) \approx f(x, y, t) + \frac{\partial f}{\partial x} \delta x + \frac{\partial f}{\partial y} \delta y + \frac{\partial f}{\partial t} \delta t \quad \boxed{2}$$

The brightness constancy constraint

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t) \quad \text{----- (1)}$$

$$I(x + \delta x, y + \delta y, t + \delta t) = I(x, y, t) + I_x \delta x + I_y \delta y + I_t \delta t \quad \text{----- (2)}$$

Subtract (1) from (2): $I_x \delta x + I_y \delta y + I_t \delta t = 0$ 1

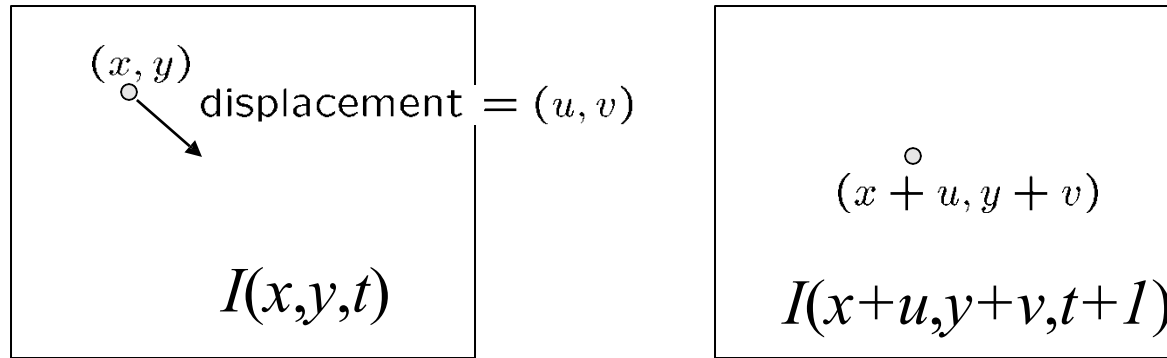
Divide by δt and take limit as $\delta t \rightarrow 0$: $I_x \frac{\partial x}{\partial t} + I_y \frac{\partial y}{\partial t} + I_t = 0$

Constraint Equation: $I_x u + I_y v + I_t = 0$ (u, v) : Optical Flow

(I_x, I_y, I_t) can be easily computed from two frames

(u, v): optical flow or feature flow

The brightness constancy constraint



- Brightness Constancy Equation:

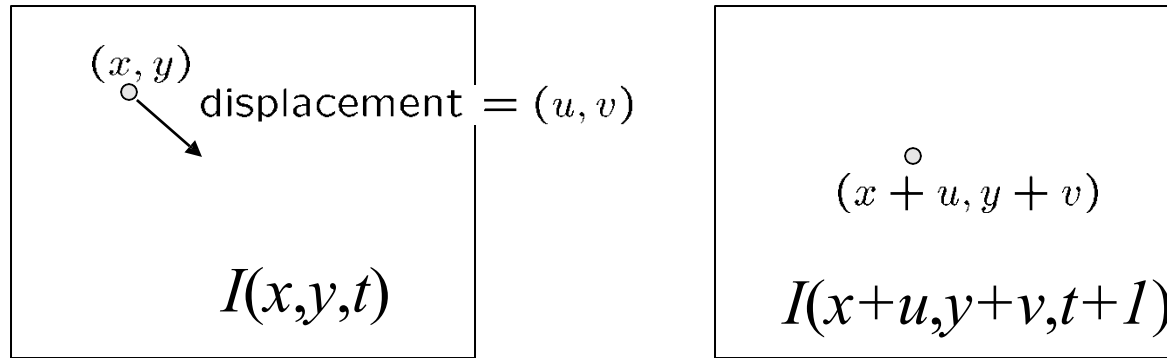
$$I(x, y, t) = I(x + u, y + v, t + 1)$$

Take Taylor expansion of $I(x + u, y + v, t + 1)$ at (x, y, t) to linearize the right side:

derivative
along x

$$I(x + u, y + v, t + 1) \approx I(x, y, t) + \boxed{I_x} \cdot u + I_y \cdot v + I_t$$

The brightness constancy constraint



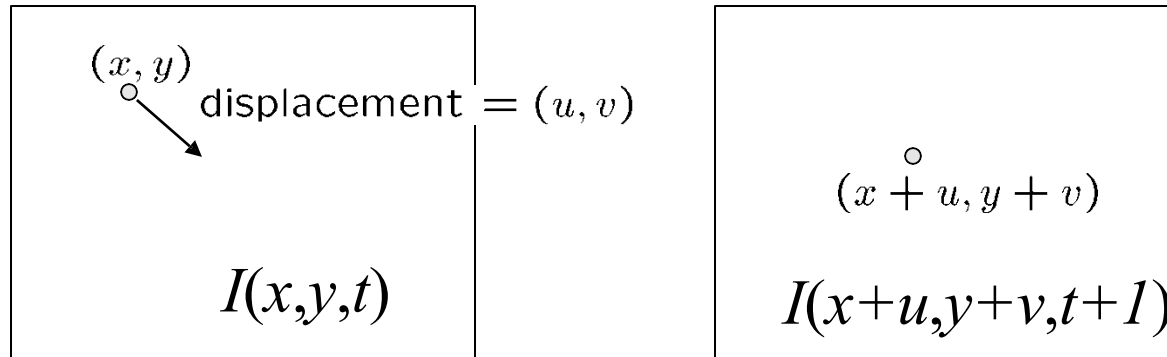
- Brightness Constancy Equation:

$$I(x, y, t) = I(x + u, y + v, t + 1)$$

Take Taylor expansion of $I(x + u, y + v, t + 1)$ at (x, y, t) to linearize the right side:

$$I(x + u, y + v, t + 1) \approx I(x, y, t) + \overset{\substack{\text{derivative} \\ \text{along } x}}{I_x} \cdot u + \overset{\substack{\text{derivative} \\ \text{along } y}}{I_y} \cdot v + I_t$$

The brightness constancy constraint



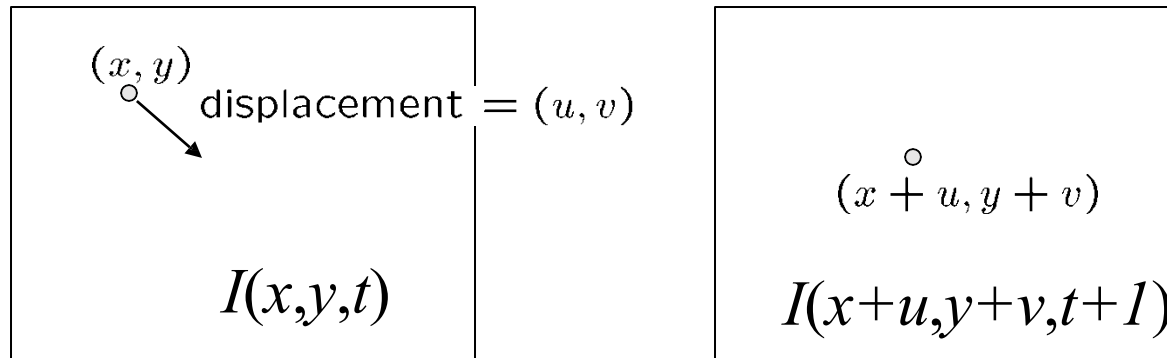
- Brightness Constancy Equation:

$$I(x, y, t) = I(x + u, y + v, t + 1)$$

Take Taylor expansion of $I(x + u, y + v, t + 1)$ at (x, y, t) to linearize the right side:

$$I(x + u, y + v, t + 1) \approx I(x, y, t) + \overset{\substack{\text{derivative} \\ \text{along } x}}{I_x} \cdot u + \overset{\substack{\text{derivative} \\ \text{along } y}}{I_y} \cdot v + \overset{\substack{\text{derivative} \\ \text{along } t}}{I_t} \quad \text{i.e., difference over frames}$$

The brightness constancy constraint



- Brightness Constancy Equation:

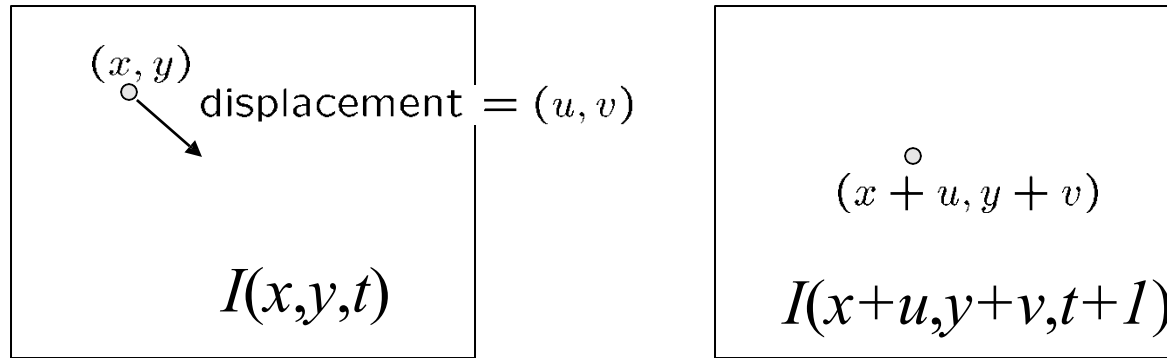
$$I(x, y, t) = I(x + u, y + v, t + 1)$$

Take Taylor expansion of $I(x + u, y + v, t + 1)$ at (x, y, t) to linearize the right side:

$$I(x + u, y + v, t + 1) \approx I(x, y, t) + \overset{\substack{\text{derivative} \\ \text{along } x}}{I_x} \cdot u + \overset{\substack{\text{derivative} \\ \text{along } y}}{I_y} \cdot v + \overset{\substack{\text{derivative} \\ \text{along } t}}{I_t} \quad \text{i.e., difference over frames}$$
$$I(x + u, y + v, t + 1) - I(x, y, t) = I_x \cdot u + I_y \cdot v + I_t$$

(u, v) : optical flow or feature flow

The brightness constancy constraint



- Brightness Constancy Equation:

$$I(x, y, t) = I(x + u, y + v, t + 1)$$

Take Taylor expansion of $I(x + u, y + v, t + 1)$ at (x, y, t) to linearize the right side:

$$I(x + u, y + v, t + 1) \approx I(x, y, t) + \overset{\substack{\text{derivative} \\ \text{along } x}}{I_x} \cdot u + \overset{\substack{\text{derivative} \\ \text{along } y}}{I_y} \cdot v + \overset{\substack{\text{derivative} \\ \text{along } t}}{I_t} \quad \text{i.e., difference over frames}$$

$$I(x + u, y + v, t + 1) - I(x, y, t) = I_x \cdot u + I_y \cdot v + I_t$$

So:
$$I_x \cdot u + I_y \cdot v + I_t \approx 0 \quad \rightarrow \quad \nabla I \cdot \begin{bmatrix} u & v \end{bmatrix}^T + I_t = 0$$

The brightness constancy constraint

Can we use this equation to recover image motion (u,v) at each pixel?

$$\nabla I \cdot [u \ v]^T + I_t = 0$$

The brightness constancy constraint

Can we use this equation to recover image motion (u,v) at each pixel?

$$\nabla I \cdot [u \ v]^T + I_t = 0$$

- How many equations and unknowns per pixel?

The brightness constancy constraint

Can we use this equation to recover image motion (u,v) at each pixel?

$$\nabla I \cdot [u \ v]^T + I_t = 0$$

- How many equations and unknowns per pixel?
 - One equation (this is a scalar equation!), two unknowns (u,v)

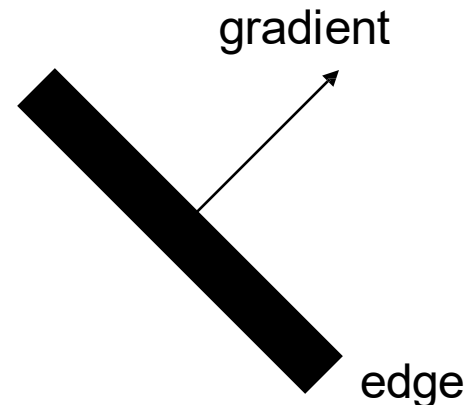
The brightness constancy constraint

Can we use this equation to recover image motion (u,v) at each pixel?

$$\nabla I \cdot [u \ v]^T + I_t = 0$$

- How many equations and unknowns per pixel?
 - One equation (this is a scalar equation!), two unknowns (u,v)

The component of the motion perpendicular to the gradient (i.e., parallel to the edge) cannot be measured



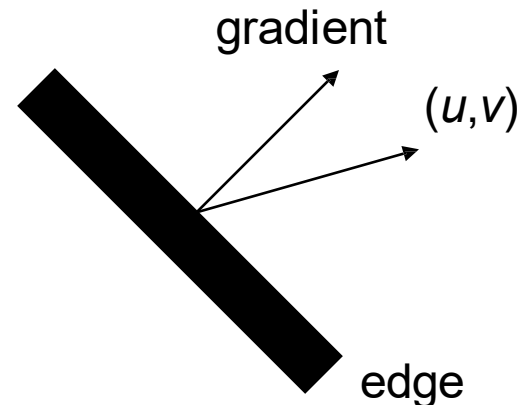
The brightness constancy constraint

Can we use this equation to recover image motion (u,v) at each pixel?

$$\nabla I \cdot [u \ v]^T + I_t = 0$$

- How many equations and unknowns per pixel?
 - One equation (this is a scalar equation!), two unknowns (u,v)

The component of the motion perpendicular to the gradient (i.e., parallel to the edge) cannot be measured



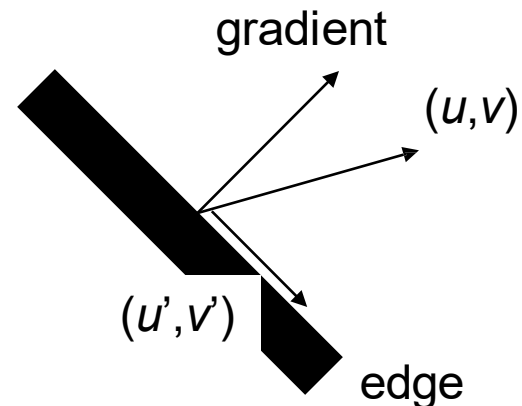
The brightness constancy constraint

Can we use this equation to recover image motion (u,v) at each pixel?

$$\nabla I \cdot [u \ v]^T + I_t = 0$$

- How many equations and unknowns per pixel?
 - One equation (this is a scalar equation!), two unknowns (u,v)

The component of the motion perpendicular to the gradient (i.e., parallel to the edge) cannot be measured



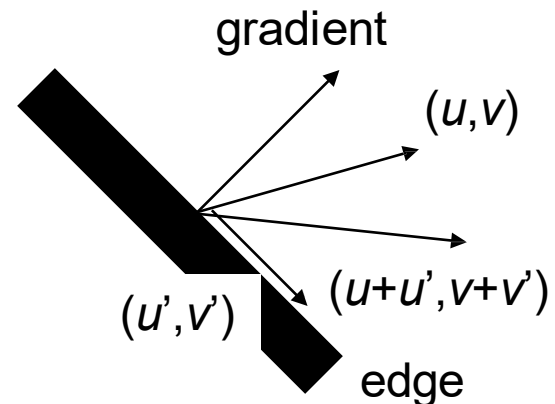
The brightness constancy constraint

Can we use this equation to recover image motion (u,v) at each pixel?

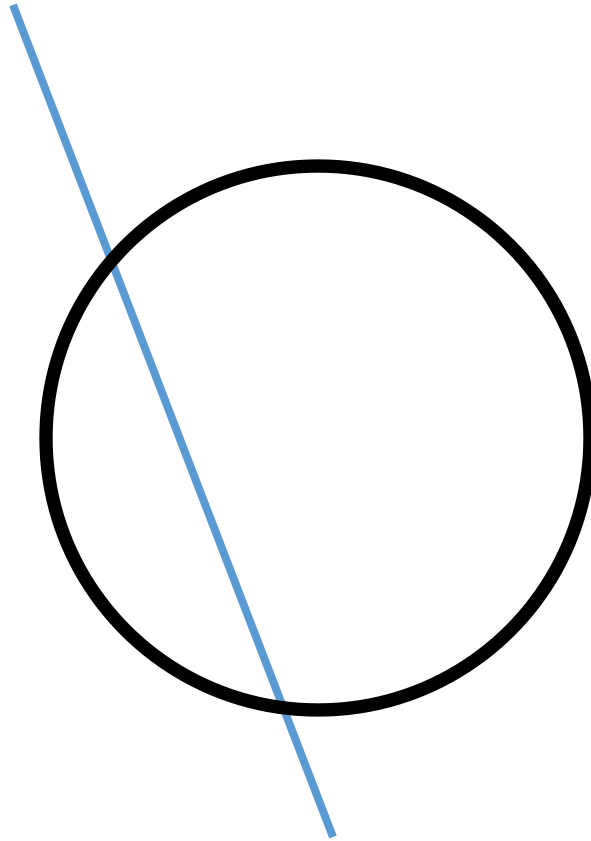
$$\nabla I \cdot [u \ v]^T + I_t = 0$$

- How many equations and unknowns per pixel?
 - One equation (this is a scalar equation!), two unknowns (u,v)

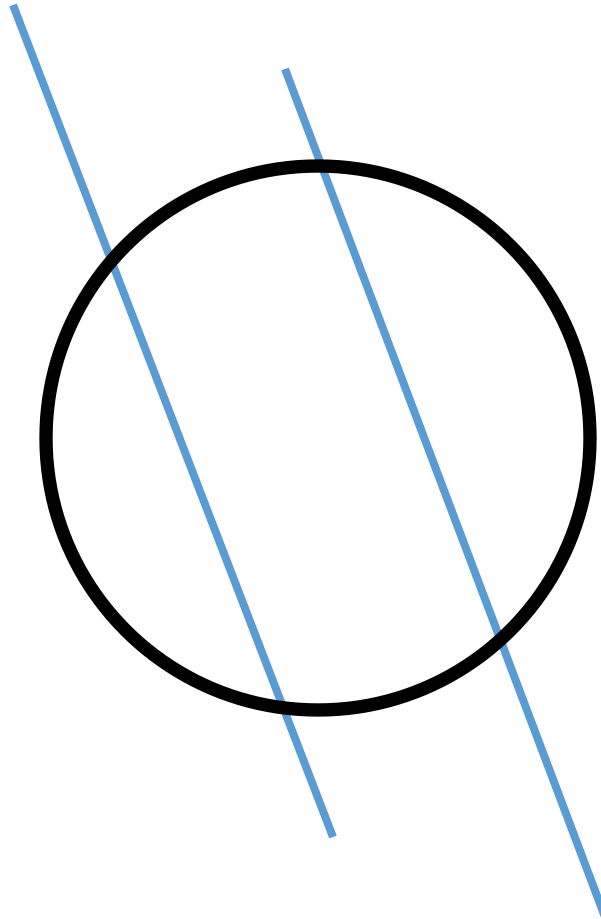
The component of the motion perpendicular to the gradient (i.e., parallel to the edge) cannot be measured



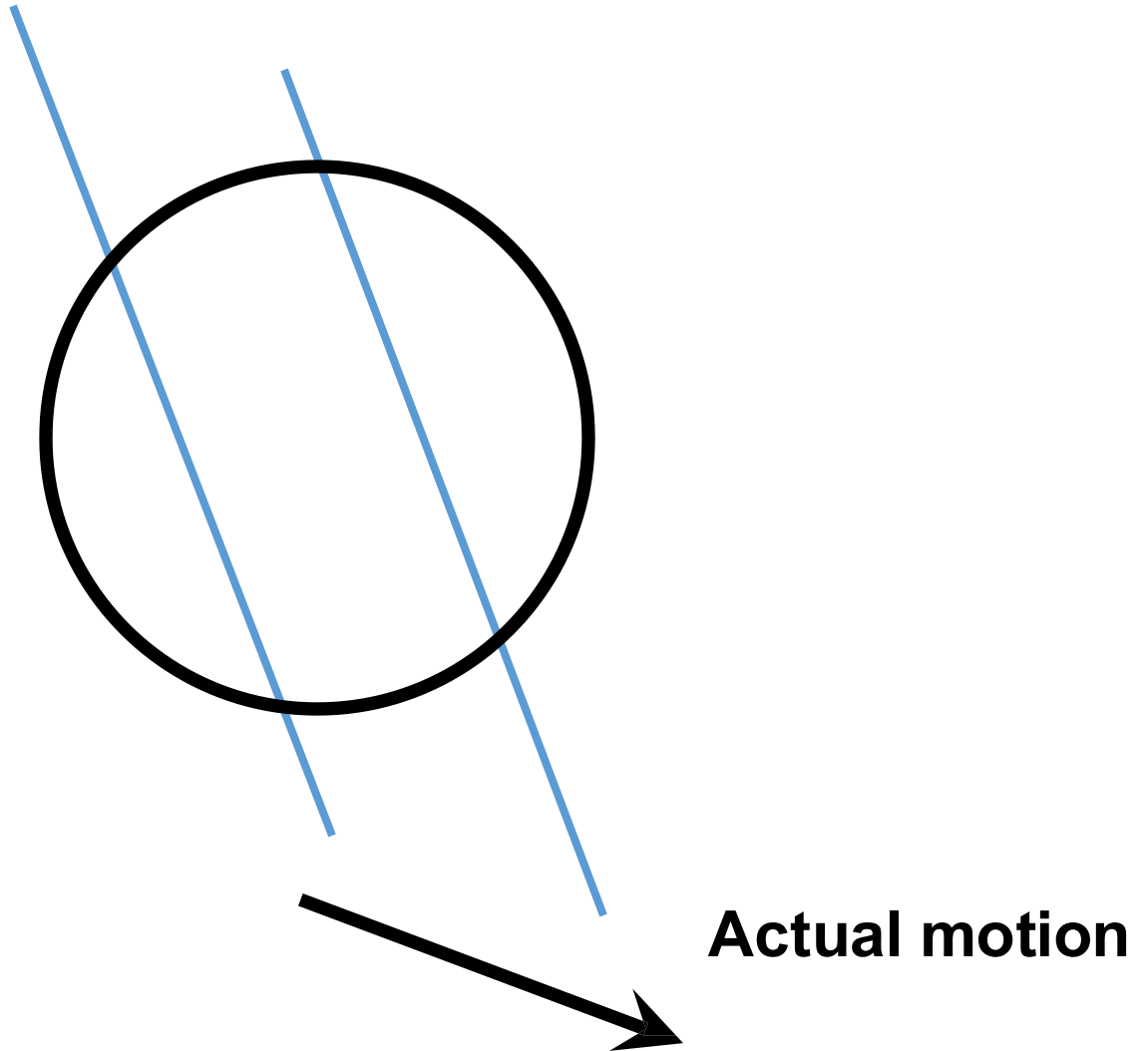
The aperture problem



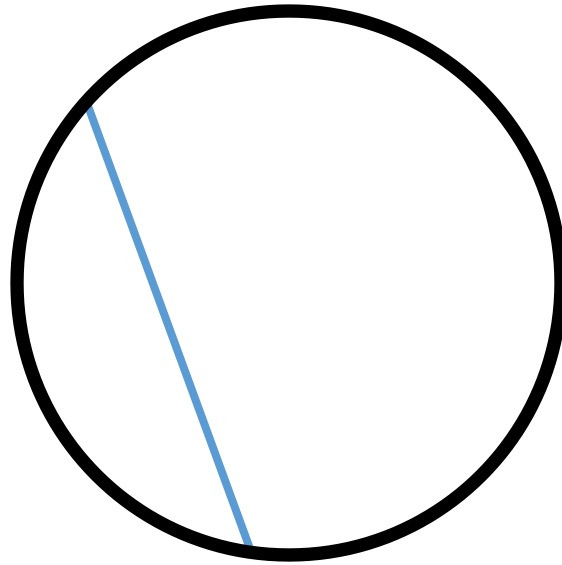
The aperture problem



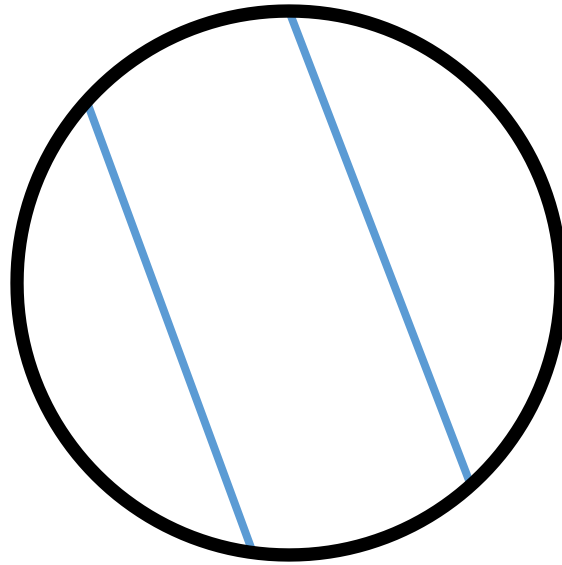
The aperture problem



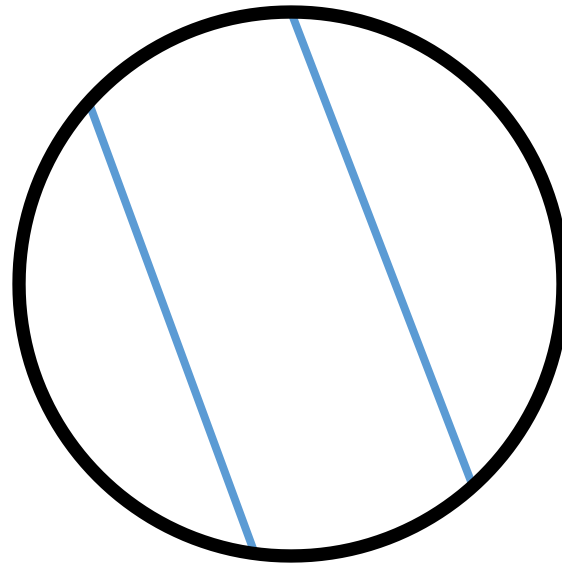
The aperture problem



The aperture problem

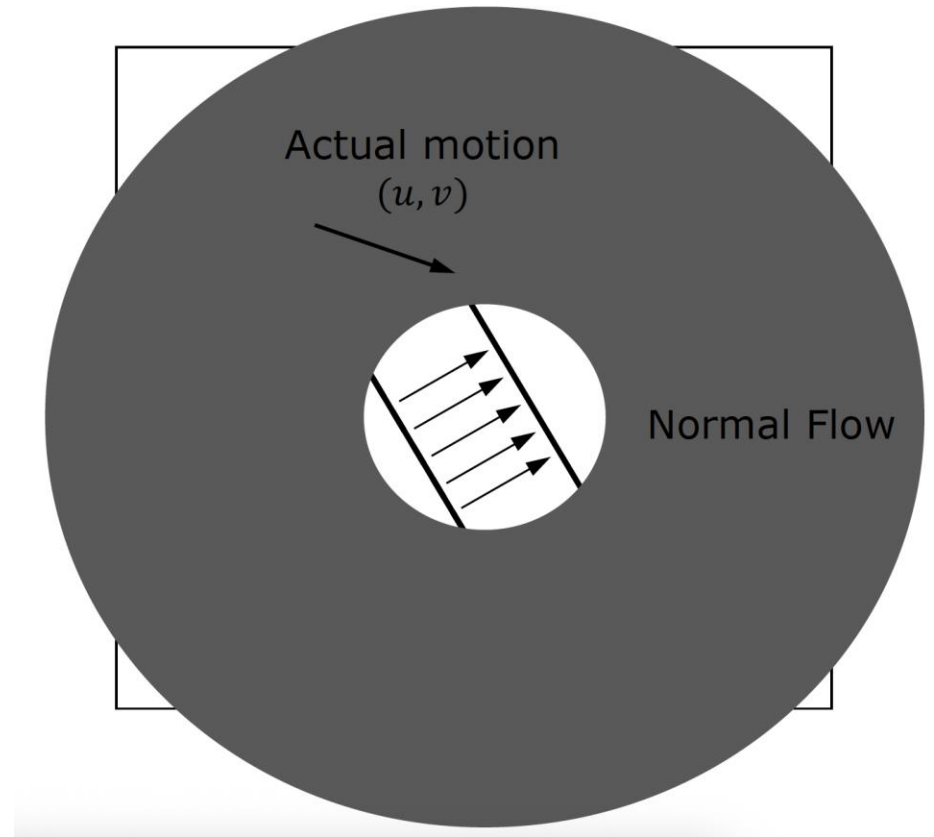
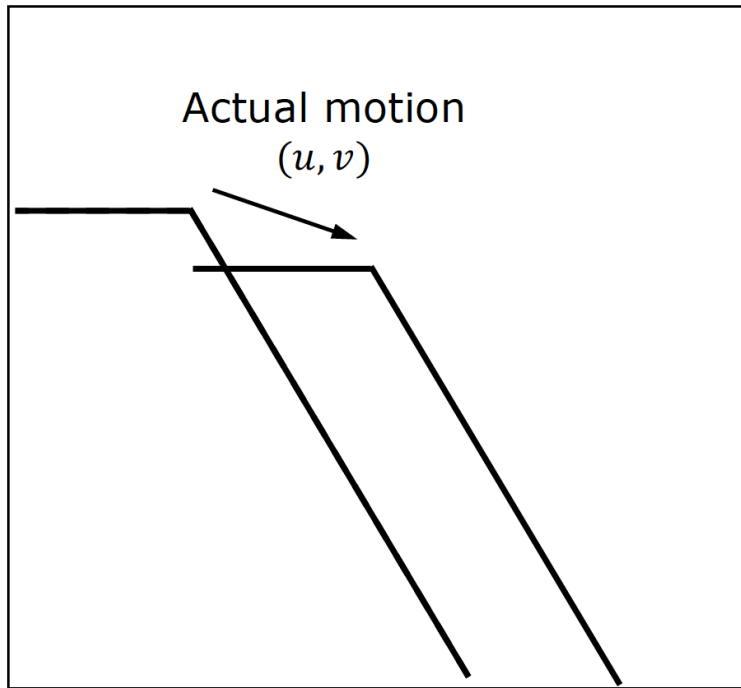


The aperture problem



Perceived motion

The aperture problem



*Locally, we can only determine
normal flow!*

Solving the ambiguity...

B. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision.
In *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 674–679, 1981.

Solving the ambiguity...

B. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision.
In *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 674–679, 1981.

- How to get more equations for a pixel?

Solving the ambiguity...

B. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 674–679, 1981.

- How to get more equations for a pixel?
- **Spatial coherence constraint**
 - Assume the pixel's neighbors have the same (u,v)
 - If we use a 5x5 window, that gives us 25 equations per pixel

That is for all points $(k, l) \in W$:

$$I_x(k, l)u + I_y(k, l)v + I_t(k, l) = 0$$

Solving the ambiguity...

B. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 674–679, 1981.

- How to get more equations for a pixel?
- **Spatial coherence constraint**
 - Assume the pixel's neighbors have the same (u,v)
 - If we use a 5x5 window, that gives us 25 equations per pixel

$$0 = I_t(\mathbf{p}_i) + \nabla I(\mathbf{p}_i) \cdot [u \ v]$$

Solving the ambiguity...

B. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision.
In *Proceedings of the International Joint Conference on Artificial Intelligence*, pp. 674–679, 1981.

- How to get more equations for a pixel?
- **Spatial coherence constraint**
 - Assume the pixel's neighbors have the same (u,v)
 - If we use a 5x5 window, that gives us 25 equations per pixel

$$0 = I_t(\mathbf{p}_i) + \nabla I(\mathbf{p}_i) \cdot [u \ v]$$

$$\begin{bmatrix} I_x(\mathbf{p}_1) & I_y(\mathbf{p}_1) \\ I_x(\mathbf{p}_2) & I_y(\mathbf{p}_2) \\ \vdots & \vdots \\ I_x(\mathbf{p}_{25}) & I_y(\mathbf{p}_{25}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(\mathbf{p}_1) \\ I_t(\mathbf{p}_2) \\ \vdots \\ I_t(\mathbf{p}_{25}) \end{bmatrix}$$

Matching patches across images

- Overconstrained linear system

$$\begin{bmatrix} I_x(\mathbf{p}_1) & I_y(\mathbf{p}_1) \\ I_x(\mathbf{p}_2) & I_y(\mathbf{p}_2) \\ \vdots & \vdots \\ I_x(\mathbf{p}_{25}) & I_y(\mathbf{p}_{25}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(\mathbf{p}_1) \\ I_t(\mathbf{p}_2) \\ \vdots \\ I_t(\mathbf{p}_{25}) \end{bmatrix} \quad \begin{matrix} A & d = b \\ 25 \times 2 & 2 \times 1 & 25 \times 1 \end{matrix}$$

Matching patches across images

- Overconstrained linear system

$$\begin{bmatrix} I_x(\mathbf{p}_1) & I_y(\mathbf{p}_1) \\ I_x(\mathbf{p}_2) & I_y(\mathbf{p}_2) \\ \vdots & \vdots \\ I_x(\mathbf{p}_{25}) & I_y(\mathbf{p}_{25}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(\mathbf{p}_1) \\ I_t(\mathbf{p}_2) \\ \vdots \\ I_t(\mathbf{p}_{25}) \end{bmatrix} \quad \begin{matrix} A & d = b \\ 25 \times 2 & 2 \times 1 & 25 \times 1 \end{matrix}$$

Least squares solution for d is given by $(A^T A) d = A^T b$

Matching patches across images

- Overconstrained linear system

$$\begin{bmatrix} I_x(p_1) & I_y(p_1) \\ I_x(p_2) & I_y(p_2) \\ \vdots & \vdots \\ I_x(p_{25}) & I_y(p_{25}) \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} I_t(p_1) \\ I_t(p_2) \\ \vdots \\ I_t(p_{25}) \end{bmatrix} \quad \begin{matrix} A & d = b \\ 25 \times 2 & 2 \times 1 & 25 \times 1 \end{matrix}$$

Least squares solution for d is given by $(A^T A) d = A^T b$

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

$A^T A$ $A^T b$

The summations are over all pixels in the $K \times K$ window

$$d = \underline{(A^T A)^{-1} A^T B}$$

Conditions for solvability

Optimal (u, v) satisfies Lucas-Kanade equation

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

$A^T A$ $A^T b$

When is this solvable? i.e., what are good points to track?

Conditions for solvability

Optimal (u, v) satisfies Lucas-Kanade equation

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

$A^T A$ $A^T b$

When is this solvable? i.e., what are good points to track?

- $A^T A$ should be invertible

Conditions for solvability

Optimal (u, v) satisfies Lucas-Kanade equation

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

$A^T A$ $A^T b$

When is this solvable? i.e., what are good points to track?

- $A^T A$ should be invertible
- $A^T A$ should not be too small eigenvalues due to noise
 - eigenvalues λ_1 and λ_2 of $A^T A$ should not be too small

Conditions for solvability

Optimal (u, v) satisfies Lucas-Kanade equation

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

$A^T A$ $A^T b$

When is this solvable? i.e., what are good points to track?

- $A^T A$ should be invertible
- $A^T A$ should not be too small due to noise
 - eigenvalues λ_1 and λ_2 of $A^T A$ should not be too small
- $A^T A$ should be well-conditioned
 - λ_1 / λ_2 should not be too large (λ_1 = larger eigenvalue)

Conditions for solvability

Optimal (u, v) satisfies Lucas-Kanade equation

$$\underbrace{\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix}}_{A^T A} \begin{bmatrix} u \\ v \end{bmatrix} = - \underbrace{\begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}}_{A^T b}$$

When is this solvable? i.e., what are good points to track?

- $A^T A$ should be invertible
- $A^T A$ should not be too small due to noise
 - eigenvalues λ_1 and λ_2 of $A^T A$ should not be too small
- $A^T A$ should be well-conditioned
 - λ_1 / λ_2 should not be too large (λ_1 = larger eigenvalue)

Does this remind you of anything?

Conditions for solvability

Optimal (u, v) satisfies Lucas-Kanade equation

$$\underbrace{\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix}}_{A^T A} \begin{bmatrix} u \\ v \end{bmatrix} = - \underbrace{\begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}}_{A^T b}$$

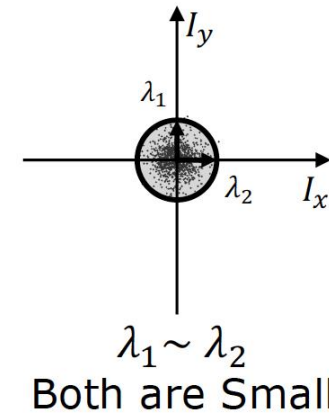
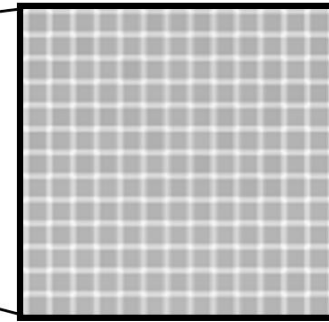
When is this solvable? i.e., what are good points to track?

- $A^T A$ should be invertible
- $A^T A$ should not be too small due to noise
 - eigenvalues λ_1 and λ_2 of $A^T A$ should not be too small
- $A^T A$ should be well-conditioned
 - λ_1 / λ_2 should not be too large (λ_1 = larger eigenvalue)

Does this remind you of anything?

Criteria for Harris corner detector

Low-texture region



- Equations for all pixels in window are more or less the same.
Cannot reliably compute flow!

$$\sum \nabla I (\nabla I)^T$$

- gradients have small magnitude
- small λ_1 , small λ_2

Edge



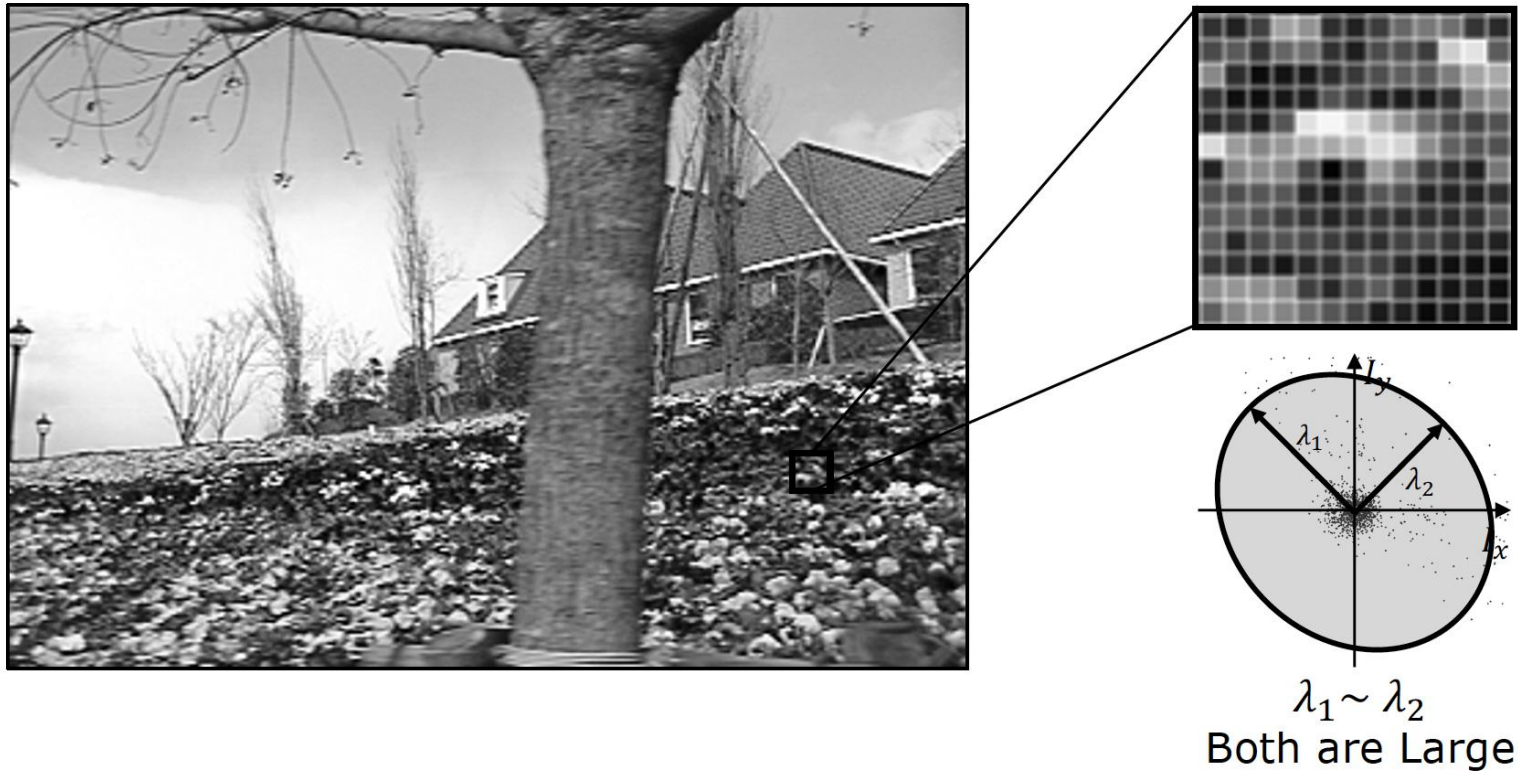
Badly conditioned. Prominent gradient in one direction. Cannot reliably compute flow! Same as Aperture Problem.

$$\sum \nabla I (\nabla I)^T$$

– gradients very large or very small

– large λ_1 , small λ_2

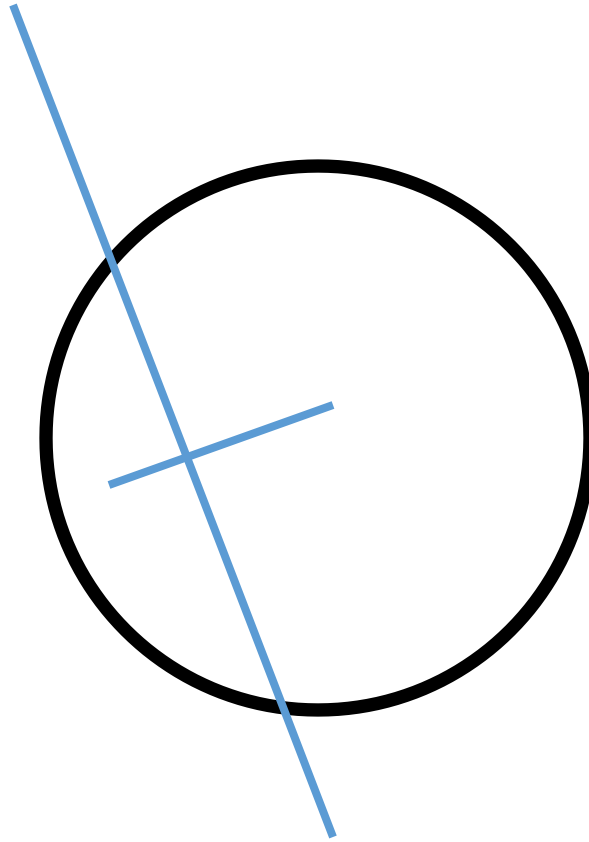
High-texture region



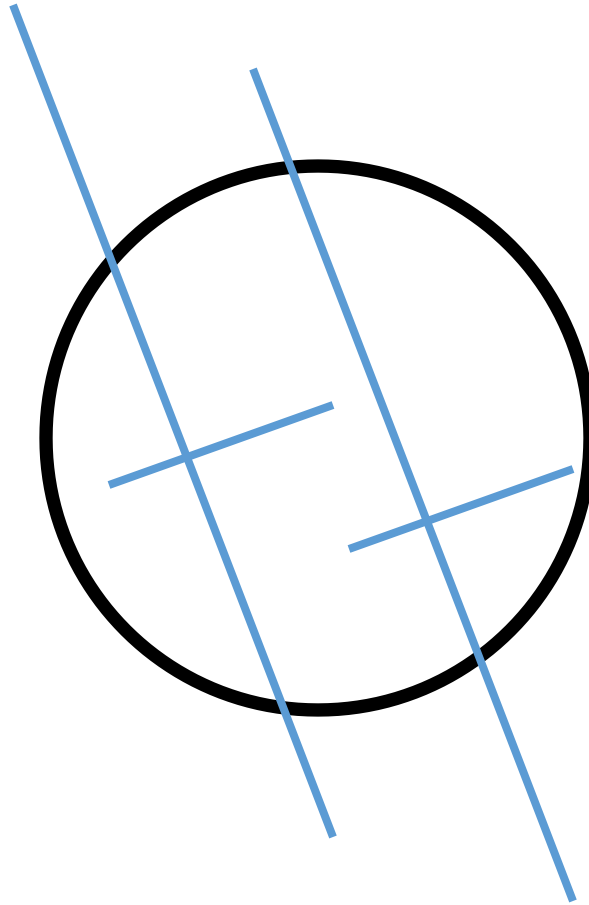
$$\sum \nabla I (\nabla I)^T$$

- gradients are different, large magnitudes
- large λ_1 , large λ_2
- Can reliably compute flow

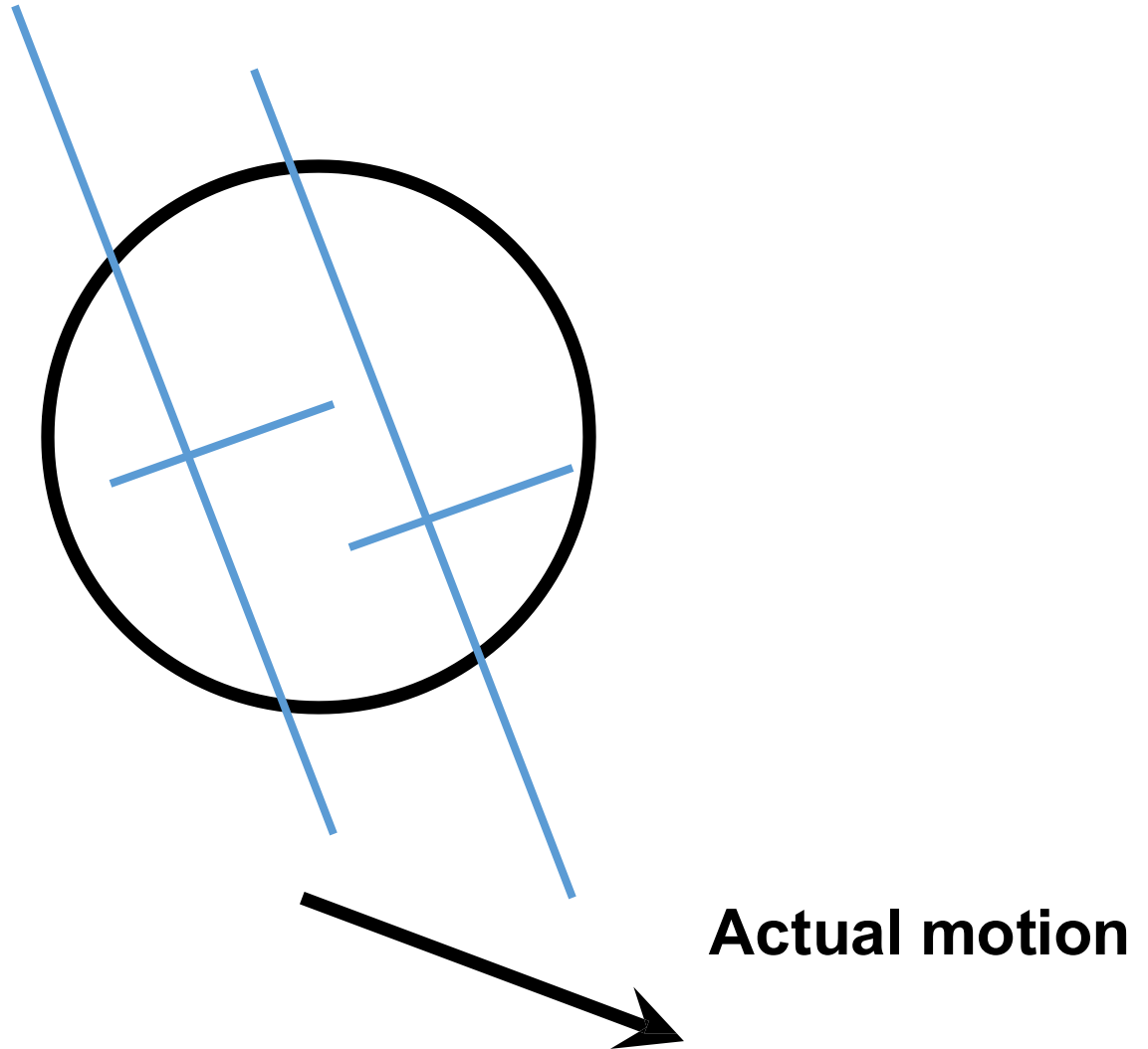
The aperture problem resolved



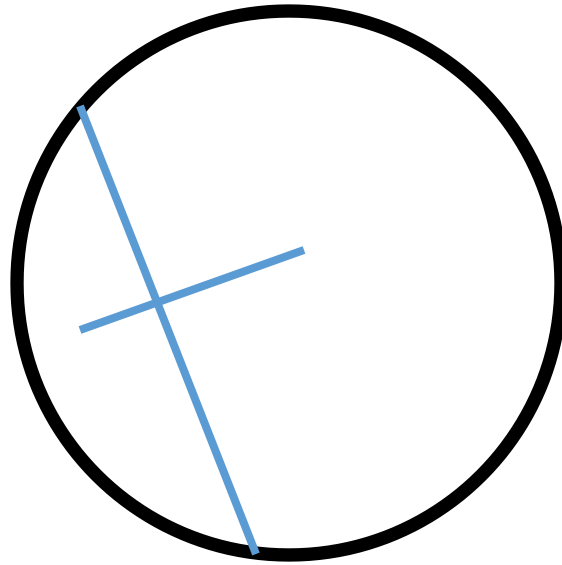
The aperture problem resolved



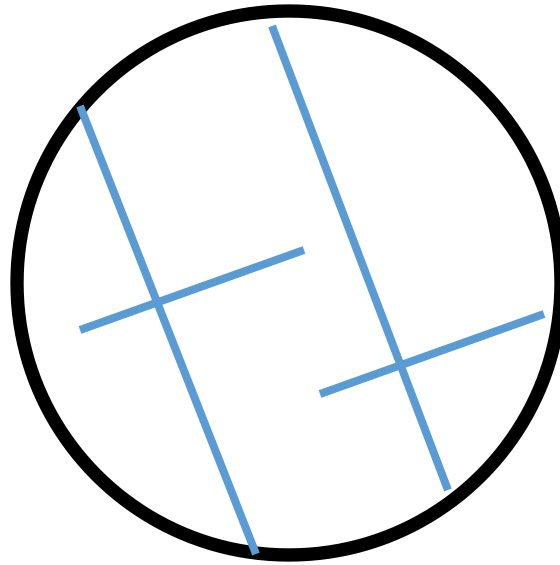
The aperture problem resolved



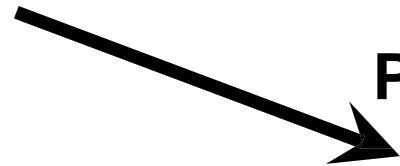
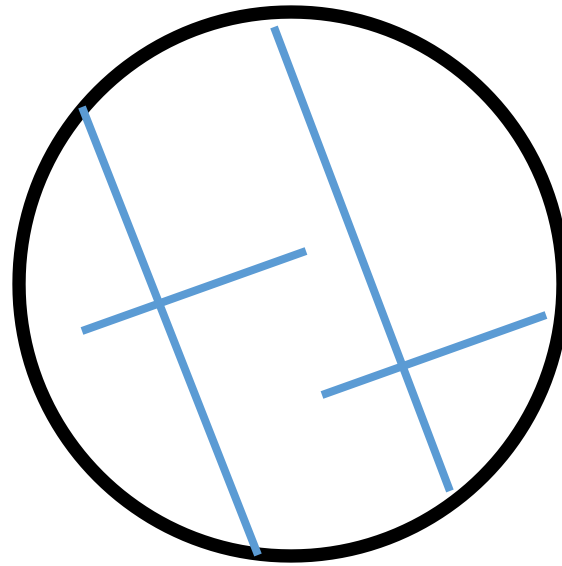
The aperture problem resolved



The aperture problem resolved



The aperture problem resolved



Perceived motion

Dealing with larger movements:
coarse-to-fine registration

What if we have large motion ?



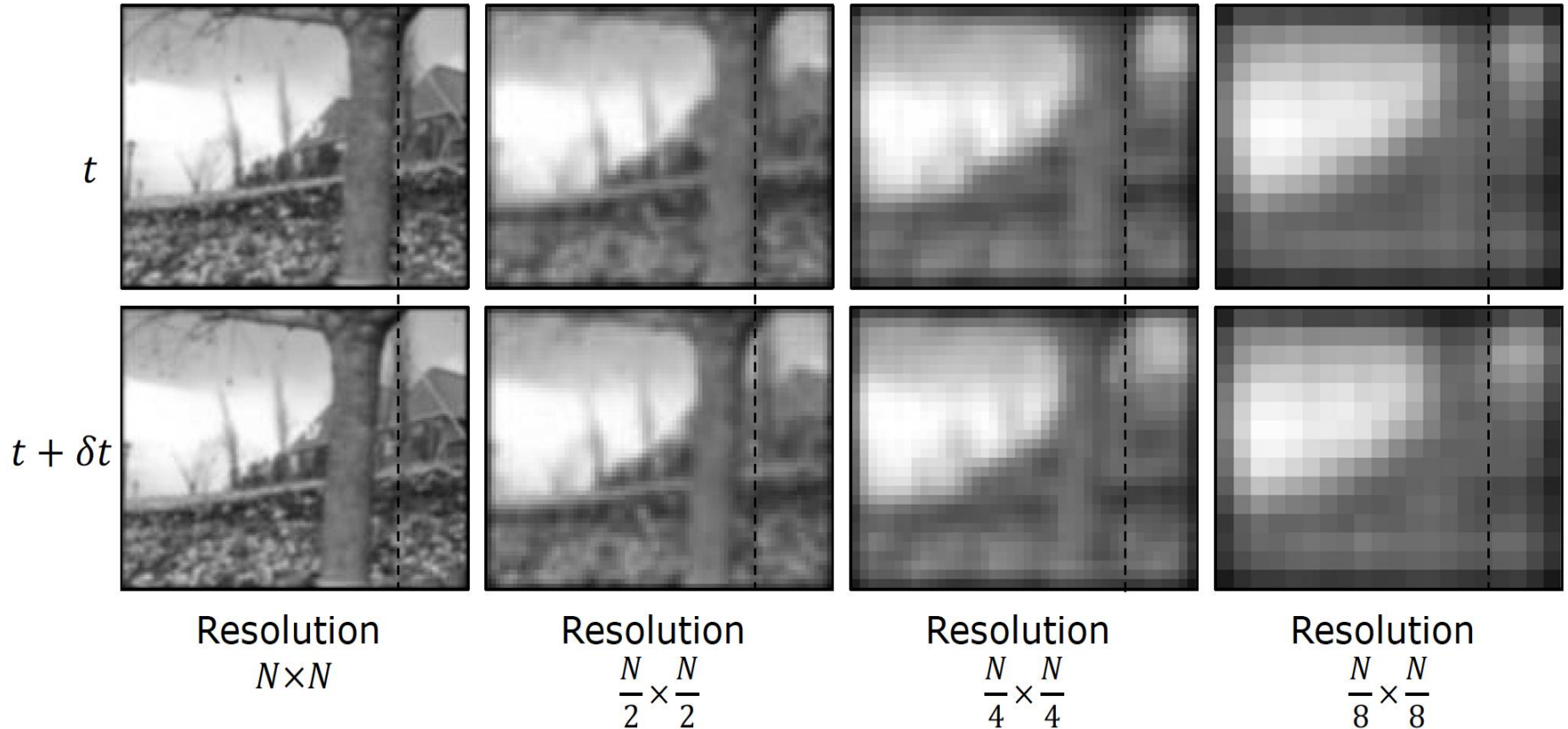
Taylor Series approximation of
 $I(x + \delta x, y + \delta y, t + \delta t)$ is not valid



Our simple linear
constraint equation not valid

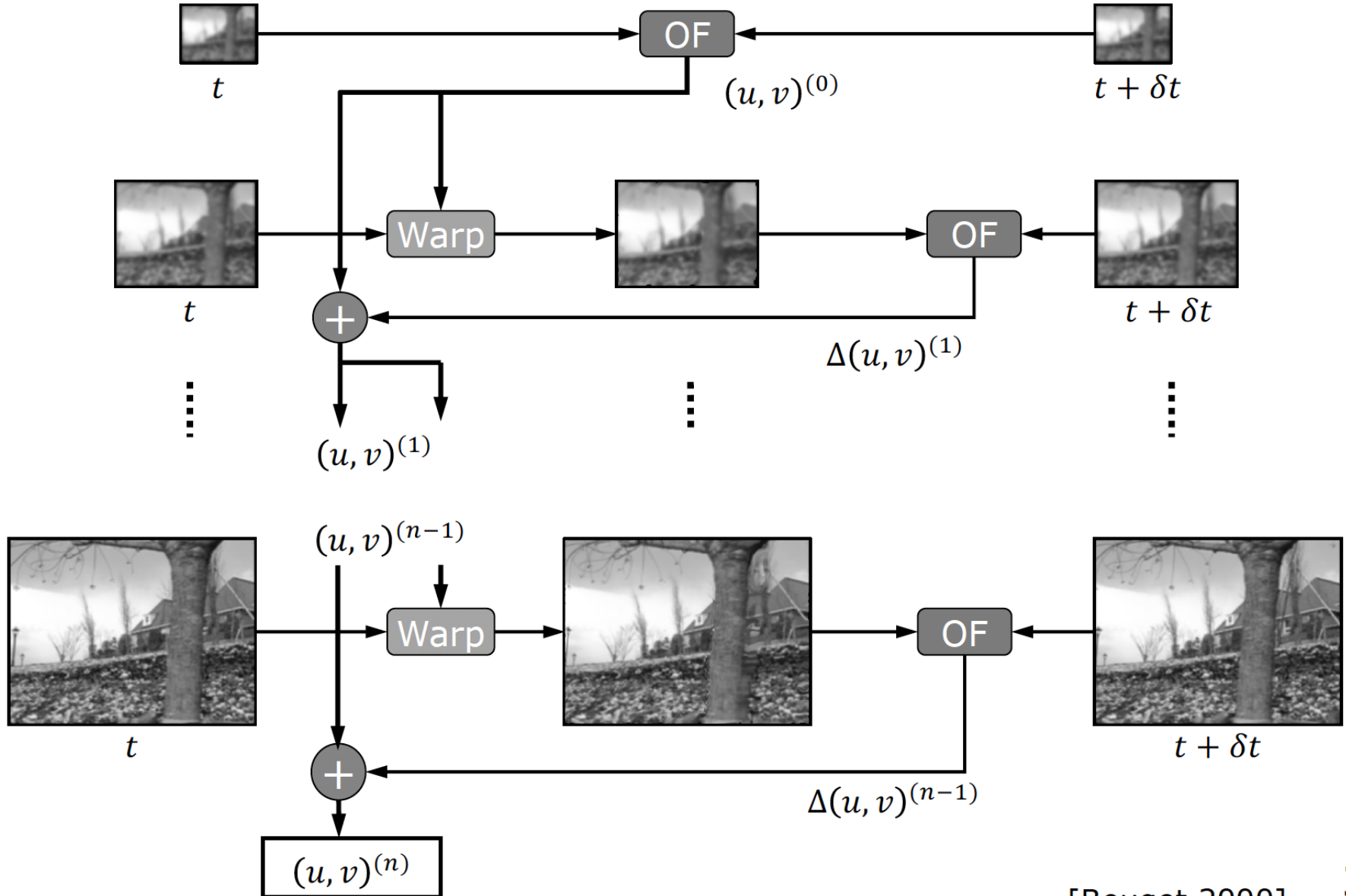
$$I_x u + I_y v + I_t \neq 0$$

Large motion: Course-to-fine Estimation

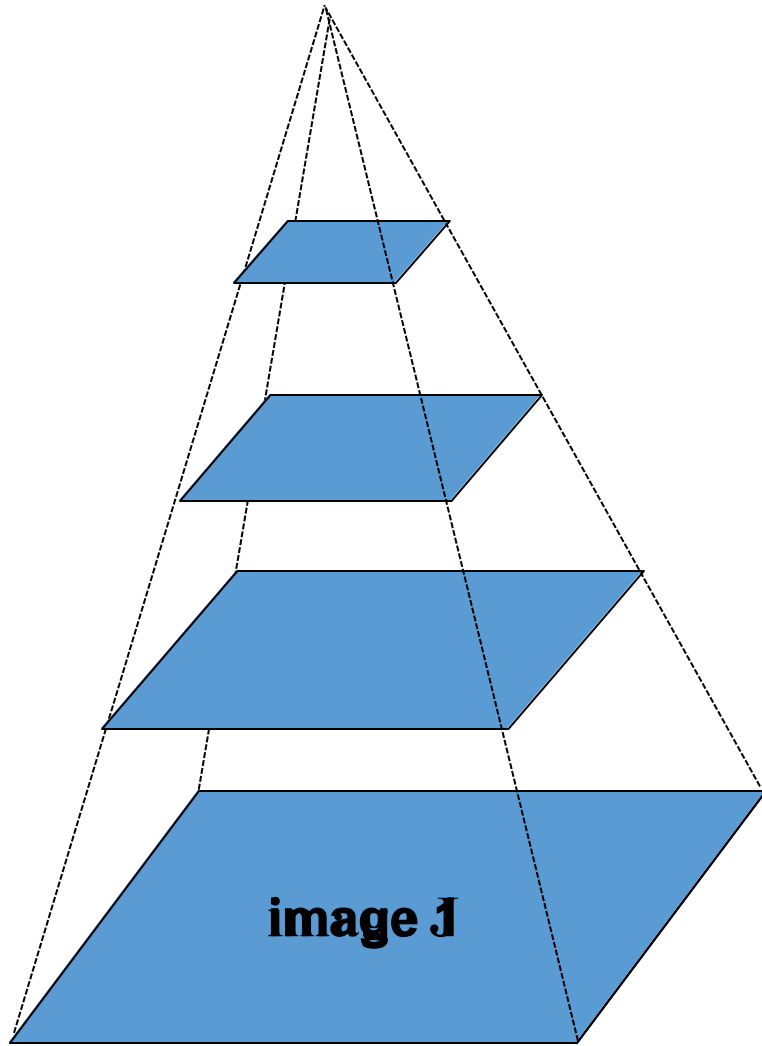


- *create a resolution pyramid*
- *At lowest resolution the motion at all pixels is less than 1 pixel.*
- *At that resolution, the optical flow constraint equation becomes valid again*

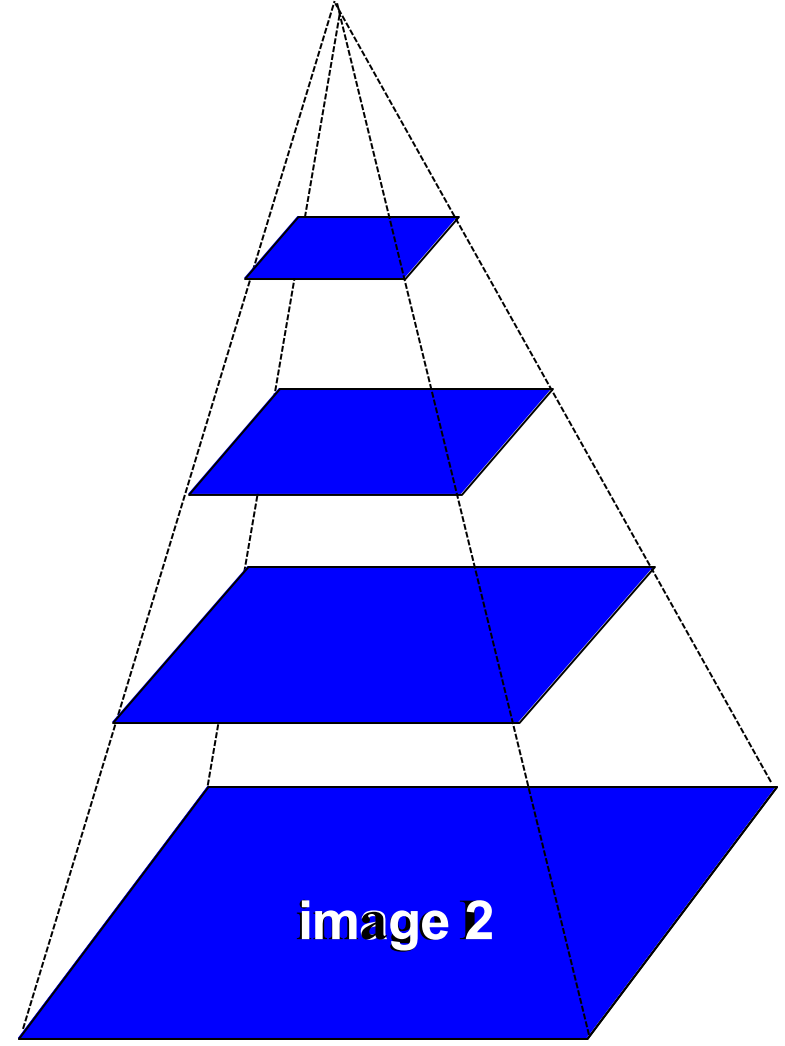
Large motion: Course-to-fine Estimation



Dealing with larger movements: coarse-to-fine registration

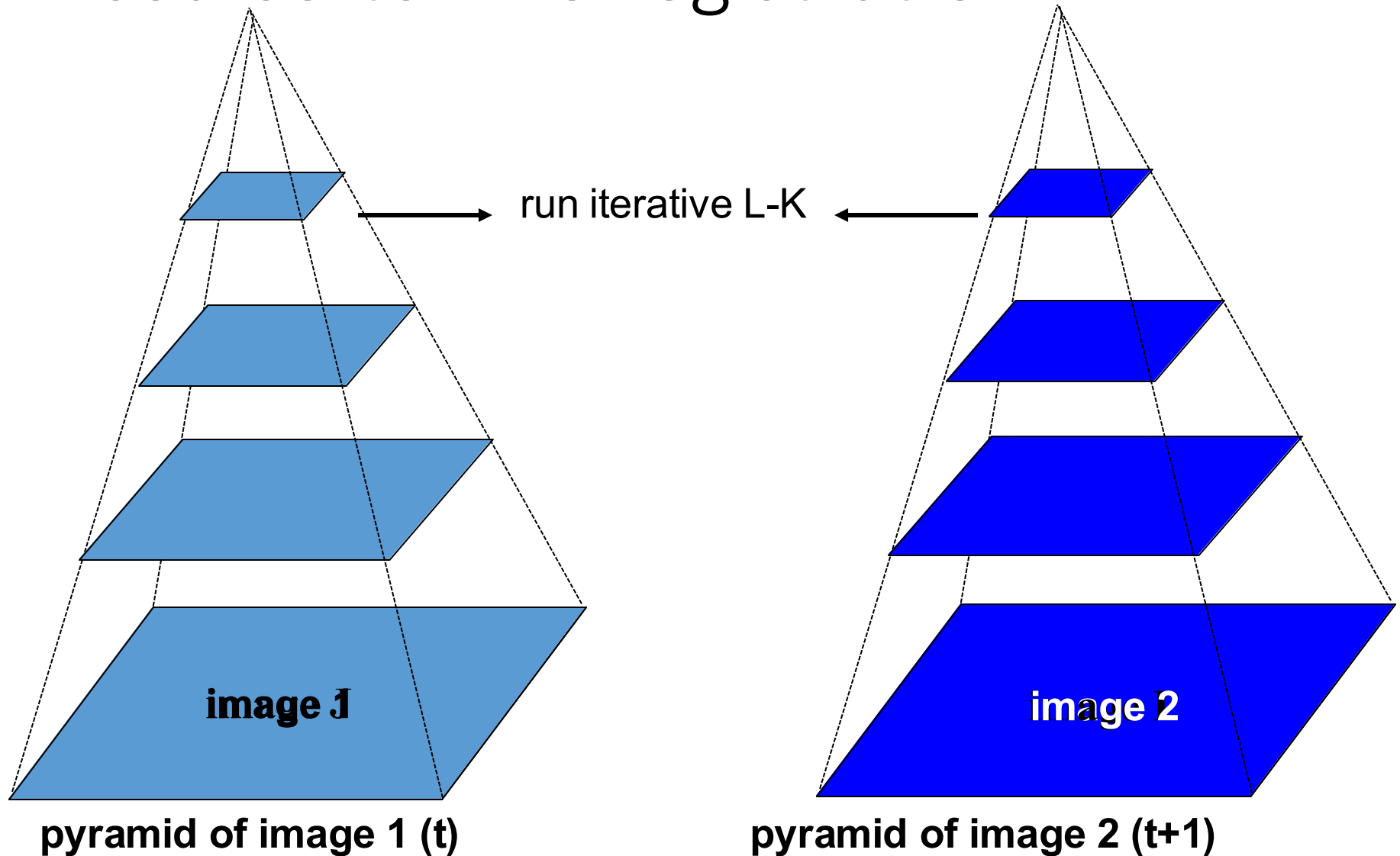


Gaussian pyramid of image 1 (t)

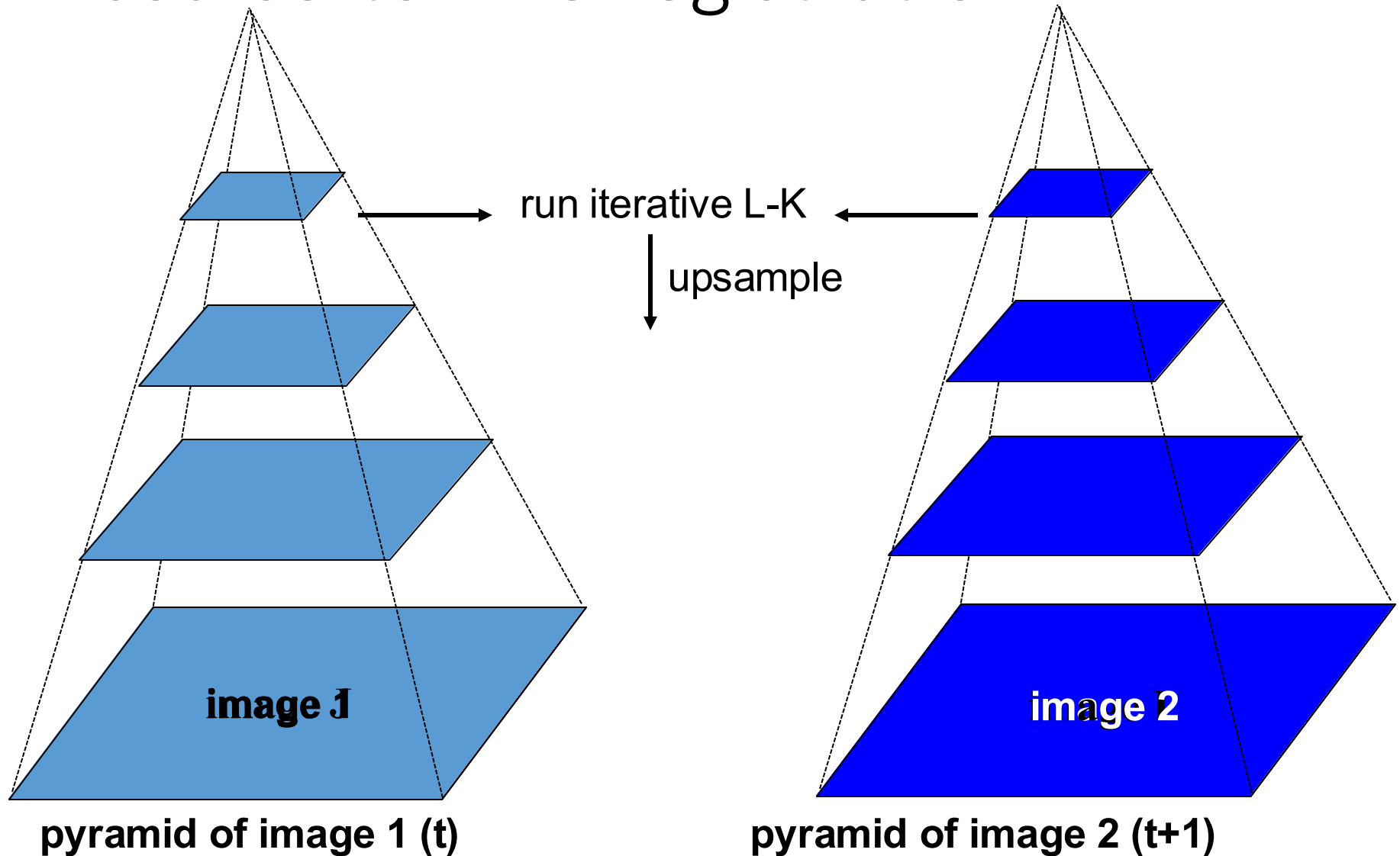


Gaussian pyramid of image 2 (t+1)

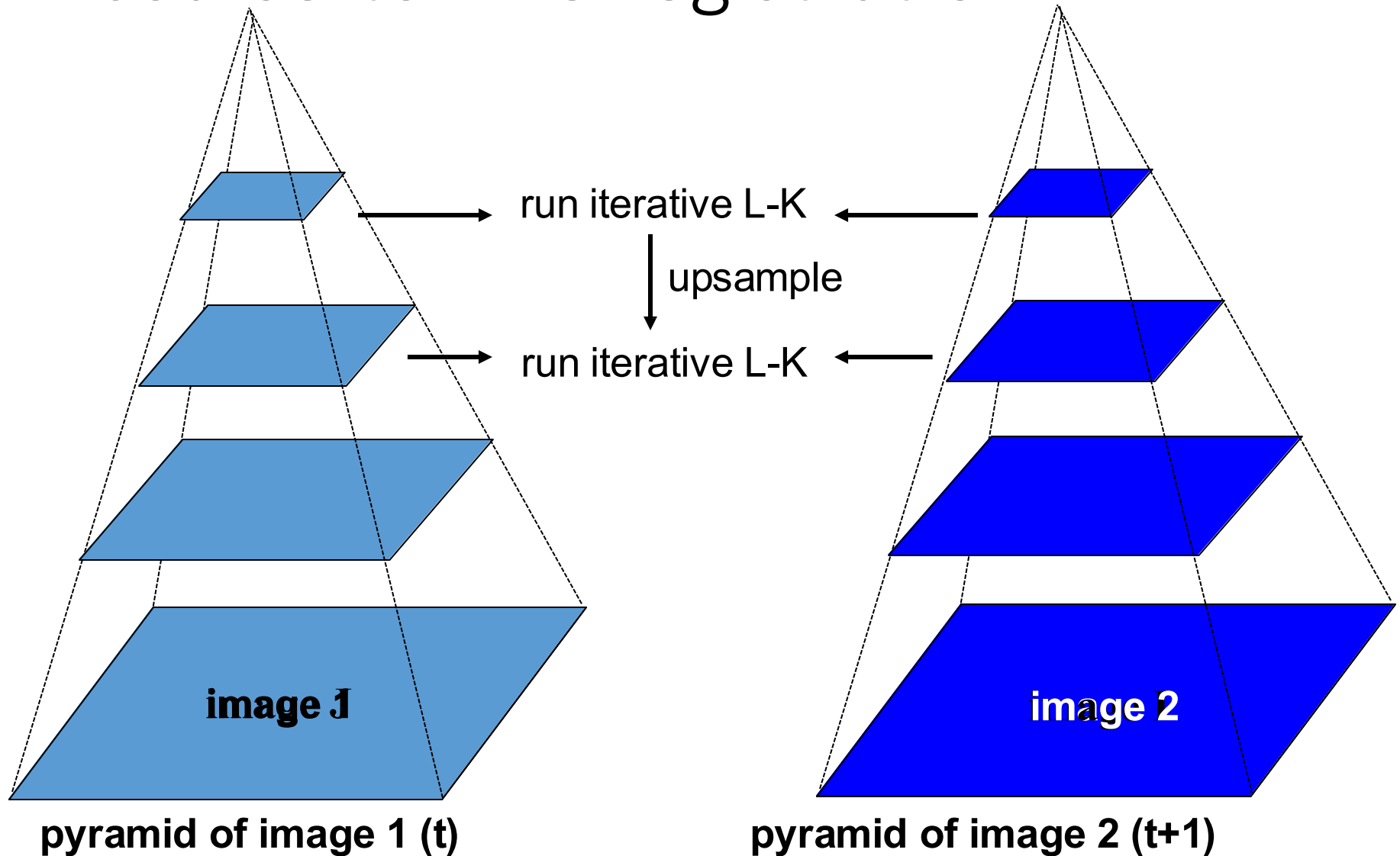
Dealing with larger movements: coarse-to-fine registration



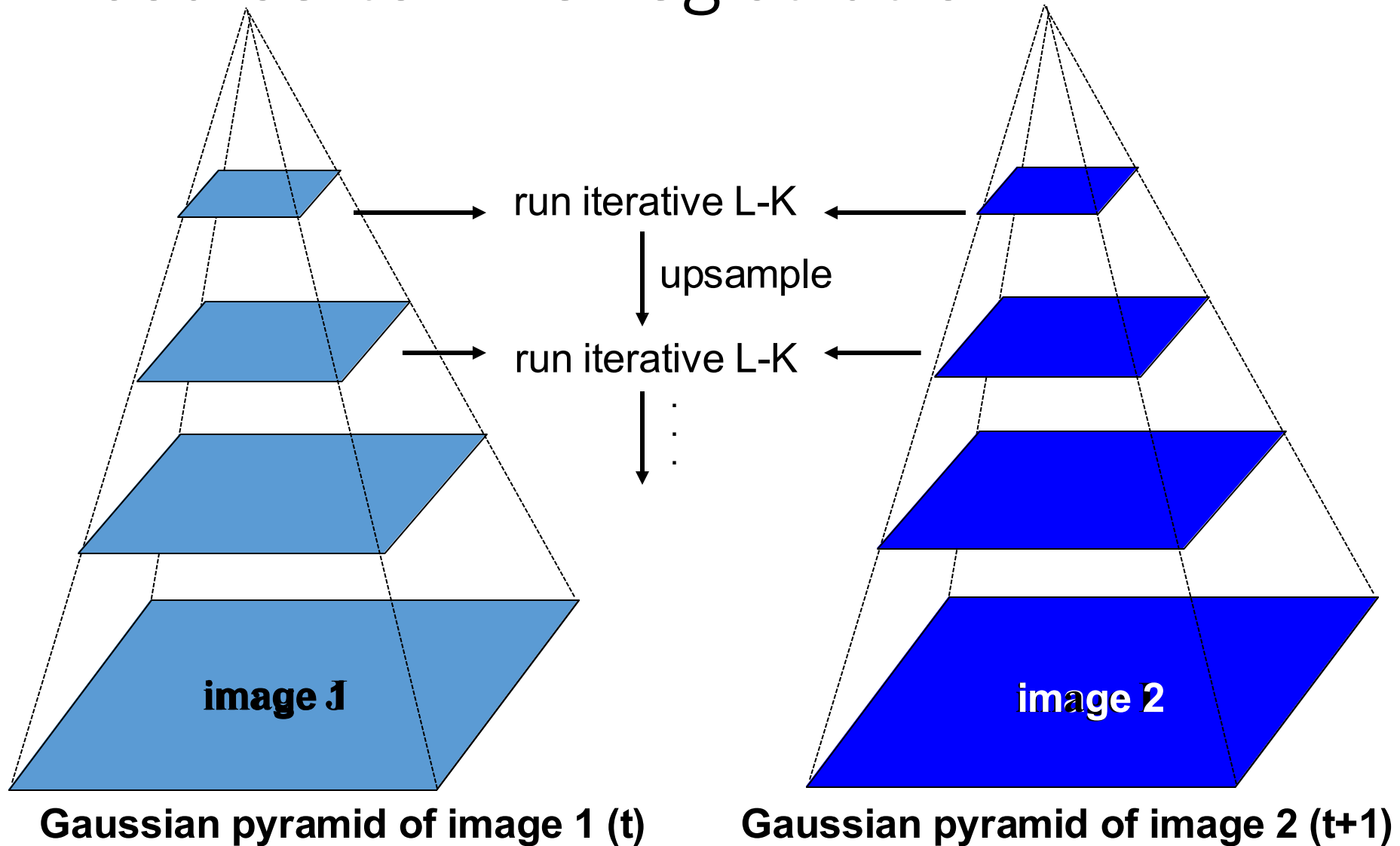
Dealing with larger movements: coarse-to-fine registration



Dealing with larger movements: coarse-to-fine registration



Dealing with larger movements: coarse-to-fine registration



Alternative Approach: Template matching



Template Window T

Image I_1 at time t



Search Window S

Image I_2 at time $t + \delta t$

For each template window T in image I_1 ,
find the corresponding match in image I_2 .

Alternative Approach: Template matching

Determine Flow using Template Matching



Template Window T

Image I_1 at time t



Search Window S

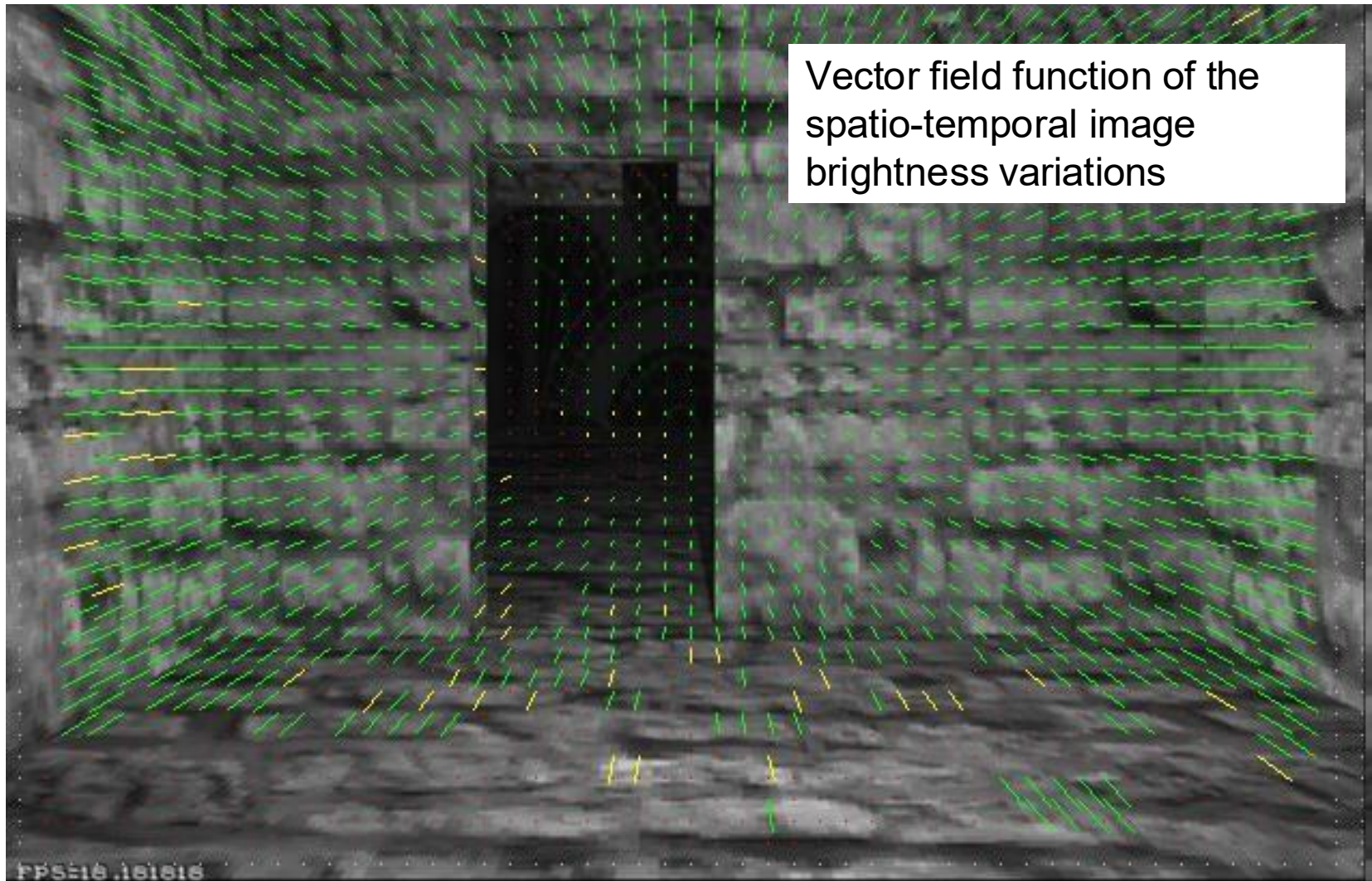
Image I_2 at time $t + \delta t$

Template matching is slow
when search window S is large.
Also, mismatches are possible.

Summary of Lucas & Kanade tracking

- Find a good point to track
- Use intensity second moment matrix and difference across frames to find displacement
- Iterate and use coarse-to-fine search to deal with larger movements
- When creating long tracks, check appearance of registered patch against appearance of initial patch to find points that have drifted

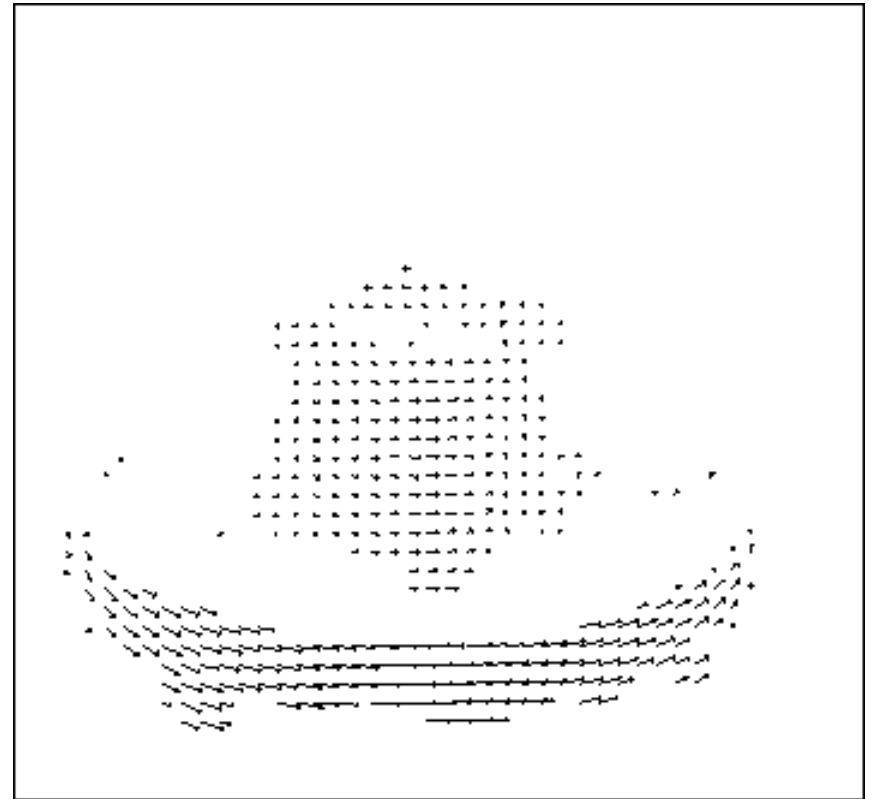
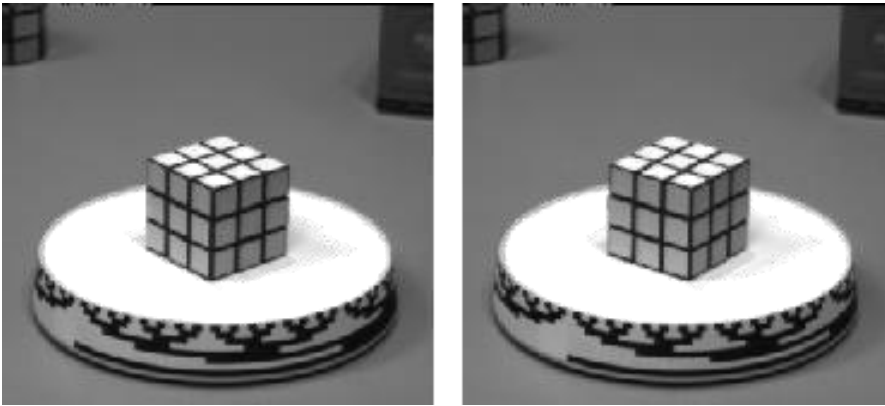
Optical flow



Picture courtesy of Selim Temizer - Learning and Intelligent Systems (LIS) Group, MIT

Motion field

- The motion field is the projection of the 3D scene motion into the image



Optical flow

- Definition: optical flow is the *apparent* motion of brightness patterns in the image

Optical flow

- Definition: optical flow is the *apparent* motion of brightness patterns in the image
- Ideally, optical flow would be the same as the motion field

Optical flow

- Definition: optical flow is the *apparent* motion of brightness patterns in the image
- Ideally, optical flow would be the same as the motion field
- Have to be careful: apparent motion can be caused by lighting changes without any actual motion

Optical flow

- Definition: optical flow is the *apparent* motion of brightness patterns in the image
- Ideally, optical flow would be the same as the motion field
- Have to be careful: apparent motion can be caused by lighting changes without any actual motion
 - Think of a uniform rotating sphere under fixed lighting vs. a stationary sphere under moving illumination

Motion Field

Image velocity of a point that is moving in the scene

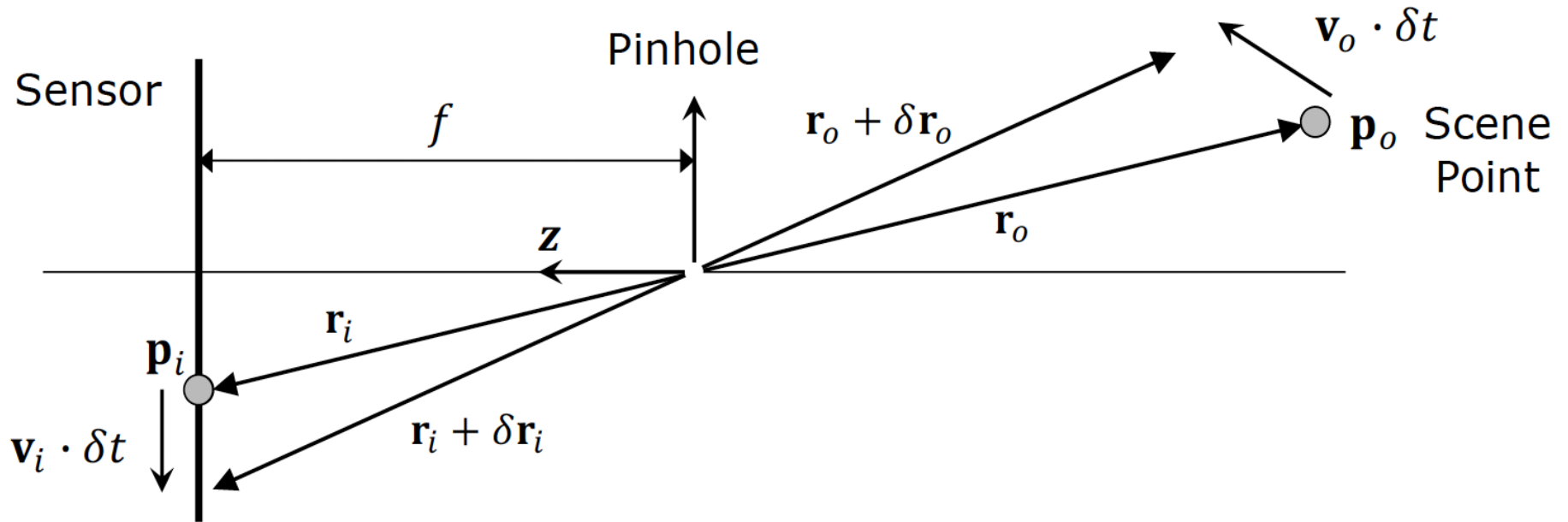
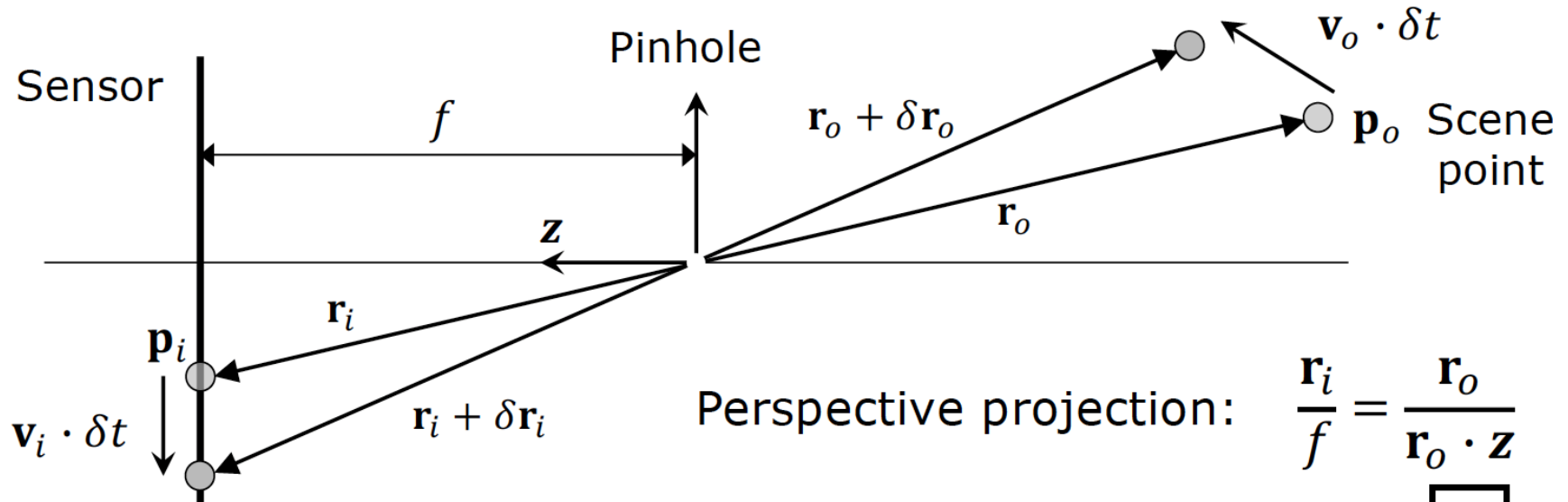


Image Point Velocity: $\mathbf{v}_i = \frac{d\mathbf{r}_i}{dt}$
(Motion Field)

Scene Point Velocity: $\mathbf{v}_o = \frac{d\mathbf{r}_o}{dt}$

Motion Field

Image velocity of a point that is moving in the scene



Perspective projection: $\frac{\mathbf{r}_i}{f} = \frac{\mathbf{r}_o}{\mathbf{r}_o \cdot \mathbf{z}}$

1

Image Point Velocity: $\mathbf{v}_i = \frac{d\mathbf{r}_i}{dt} = f \frac{(\mathbf{r}_o \cdot \mathbf{z})\mathbf{v}_o - (\mathbf{v}_o \cdot \mathbf{z})\mathbf{r}_o}{(\mathbf{r}_o \cdot \mathbf{z})^2}$
(Motion Field)

$$\mathbf{v}_i = f \frac{(\mathbf{r}_o \times \mathbf{v}_o) \times \mathbf{z}}{(\mathbf{r}_o \cdot \mathbf{z})^2}$$

2

Optical flow

Motion of brightness patterns in the image



Image Sequence
(2 frames)

Optical flow

Motion of brightness patterns in the image



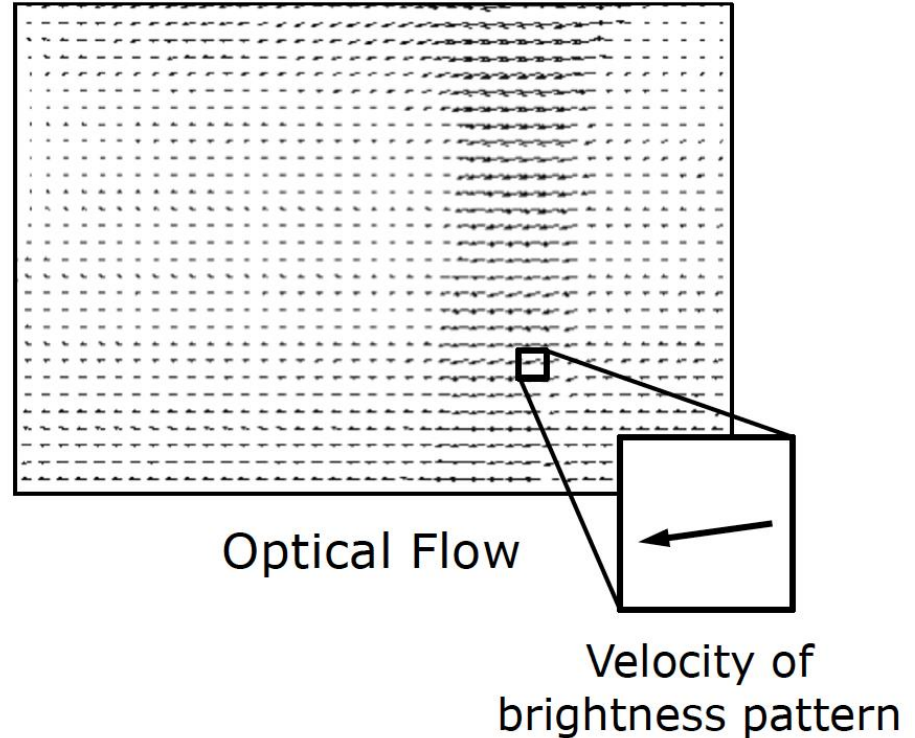
Image Sequence
(2 frames)

Optical flow

Motion of brightness patterns in the image



Image Sequence
(2 frames)



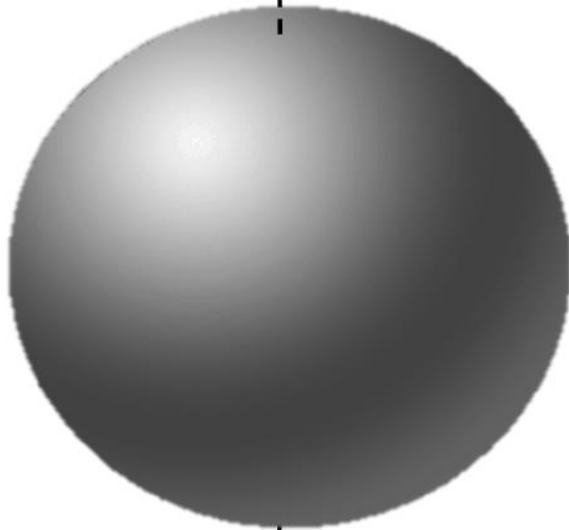
Optical Flow

Velocity of
brightness pattern

Ideally, Optical Flow = Motion Field

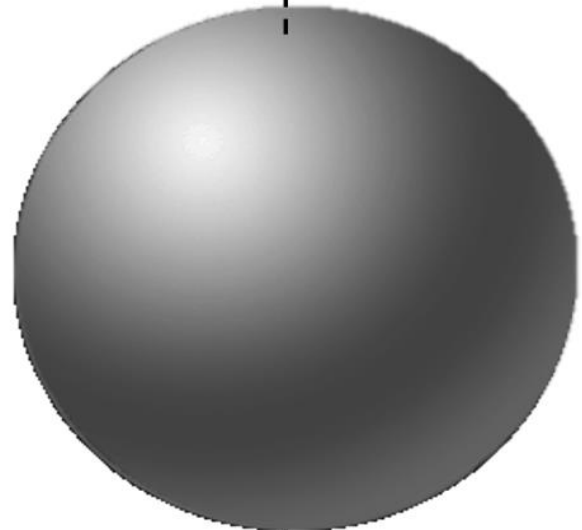
Optical flow

When is Optical Flow $\neq$ Motion Field?



Spinning Sphere
Stationary Light Source

Motion Field exists
But no Optical Flow



Stationary Sphere
Moving Light Source

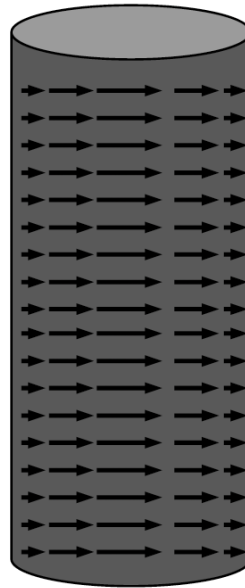
No Motion Field exists
But there is Optical Flow

Optical flow

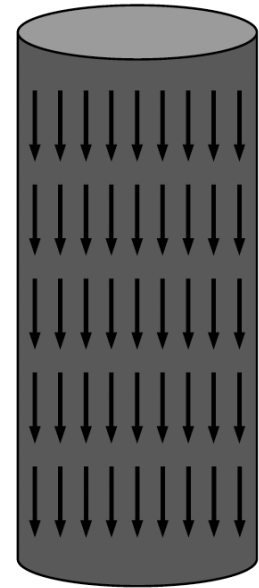
When is Optical Flow $\neq$ Motion Field?



Barber Pole
Illusion

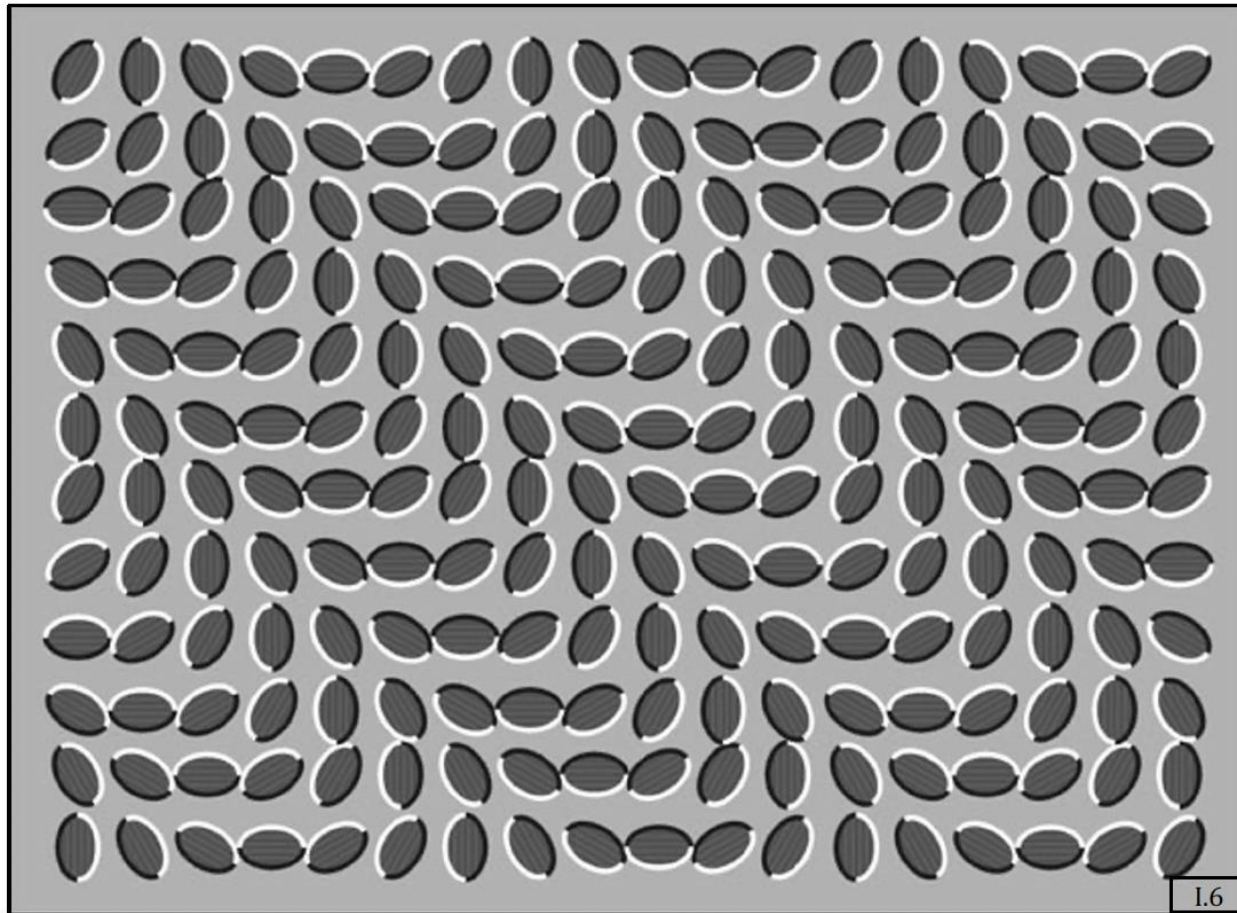


Motion Field



Optical Flow

Motion Illusions

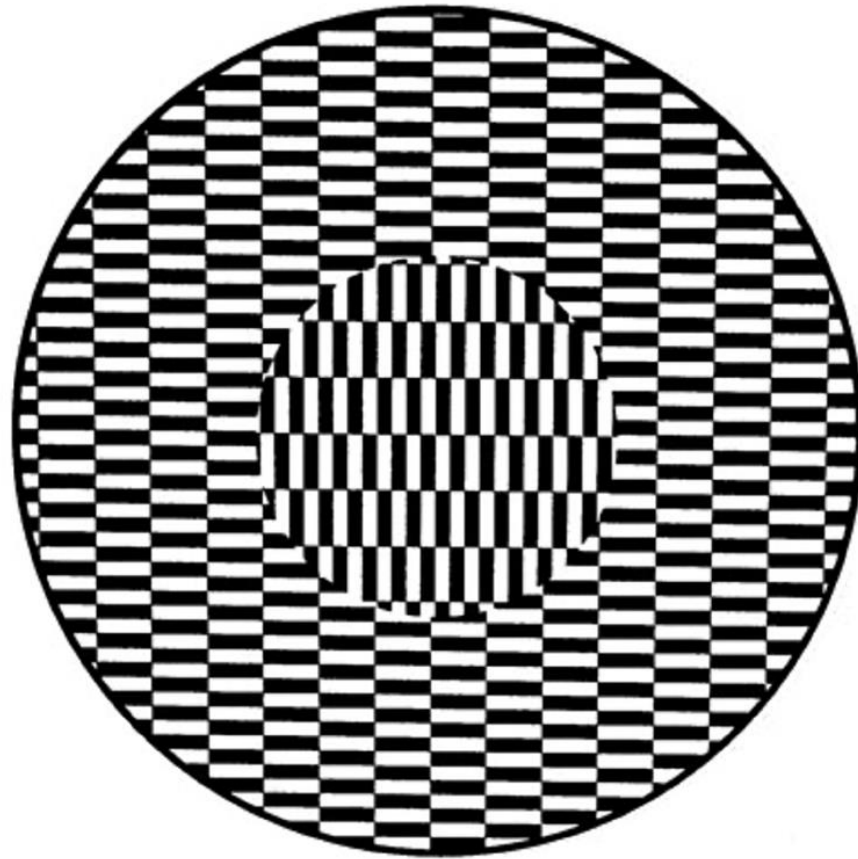


Donguri Wave Illusion

EYE AND BRAIN

- *Our visual system detects optical flow even though there is no motion field*

Motion Illusions



Ouchi Pattern

EYE AND BRAIN

- *inner disc appears to move with respect to the outer ring.*
- *There is no motion field, but we perceive optical flow.*

Lucas-Kanade Optical Flow

- Same as Lucas-Kanade feature tracking, but for each pixel
 - As we saw, works better for textured pixels
- Operations can be done one frame at a time, rather than pixel by pixel
 - Efficient

Errors in Lucas-Kanade

Errors in Lucas-Kanade

- The motion is large

Errors in Lucas-Kanade

- The motion is large
 - Possible Fix: Keypoint matching

Errors in Lucas-Kanade

- The motion is large
 - Possible Fix: Keypoint matching
- A point does not move like its neighbors

Errors in Lucas-Kanade

- The motion is large
 - Possible Fix: Keypoint matching
- A point does not move like its neighbors
 - Possible Fix: Region-based matching

Errors in Lucas-Kanade

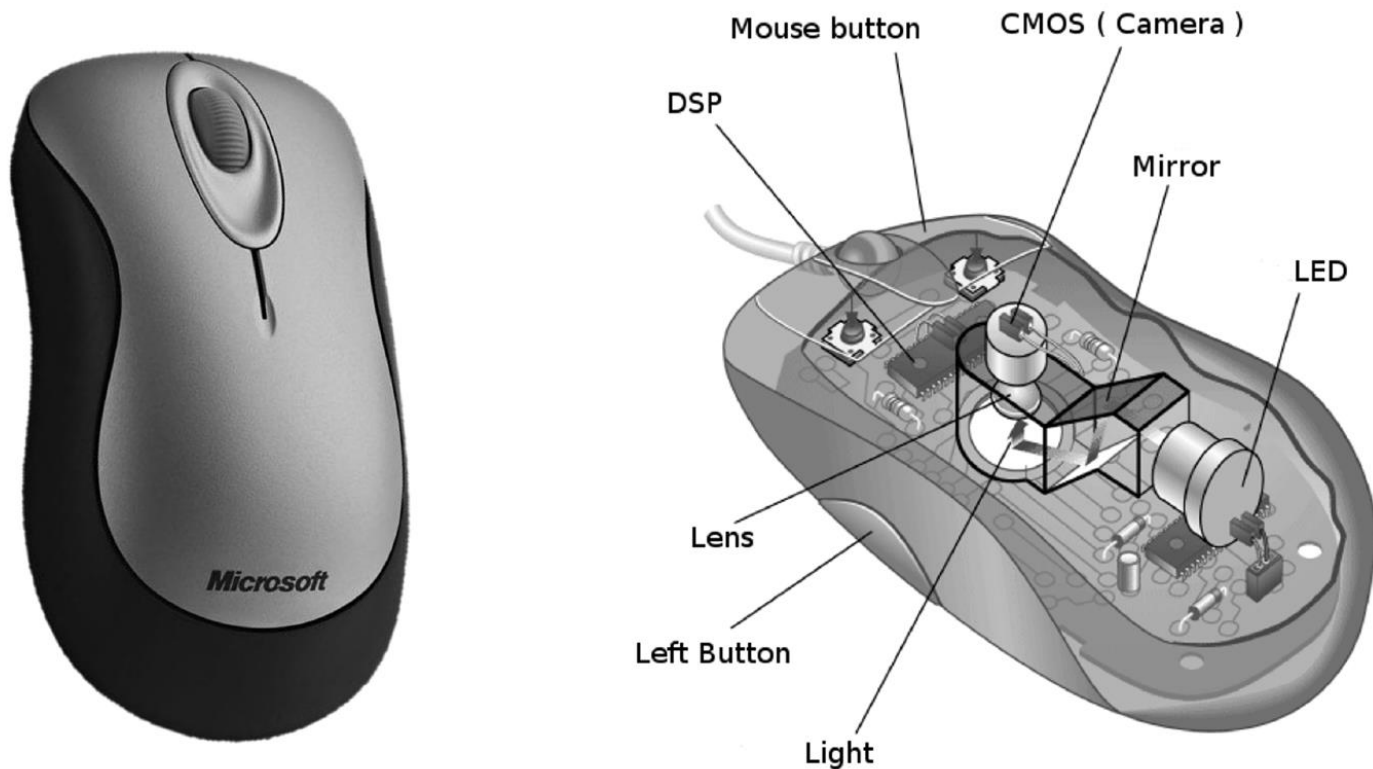
- The motion is large
 - Possible Fix: Keypoint matching
- A point does not move like its neighbors
 - Possible Fix: Region-based matching
- Brightness constancy does not hold

Errors in Lucas-Kanade

- The motion is large
 - Possible Fix: Keypoint matching
- A point does not move like its neighbors
 - Possible Fix: Region-based matching
- Brightness constancy does not hold
 - Possible Fix: Gradient constancy

Optical Flow: Applications

Optical Mouse



I.3

Estimating Mouse Movements

Traffic Monitoring



Finding Velocities of Vehicles

Videos Retiming



Optical Flow is used to determine the intermediate frames to produce slow-motion effect.

Videos Retiming



Captured Video



Stabilized Video

Optical Flow is used to remove camera shake.

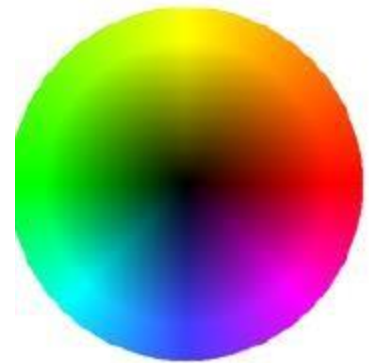
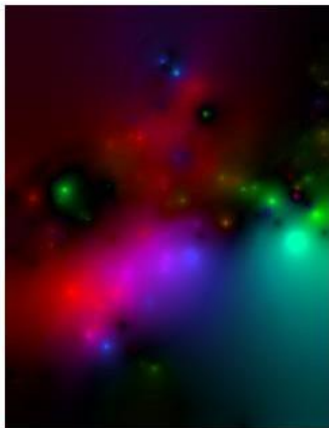
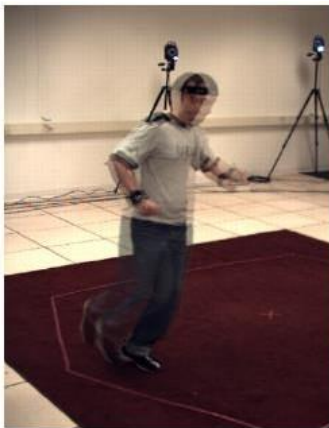
Face Tracking



Tracking of facial features

State-of-the-art optical flow

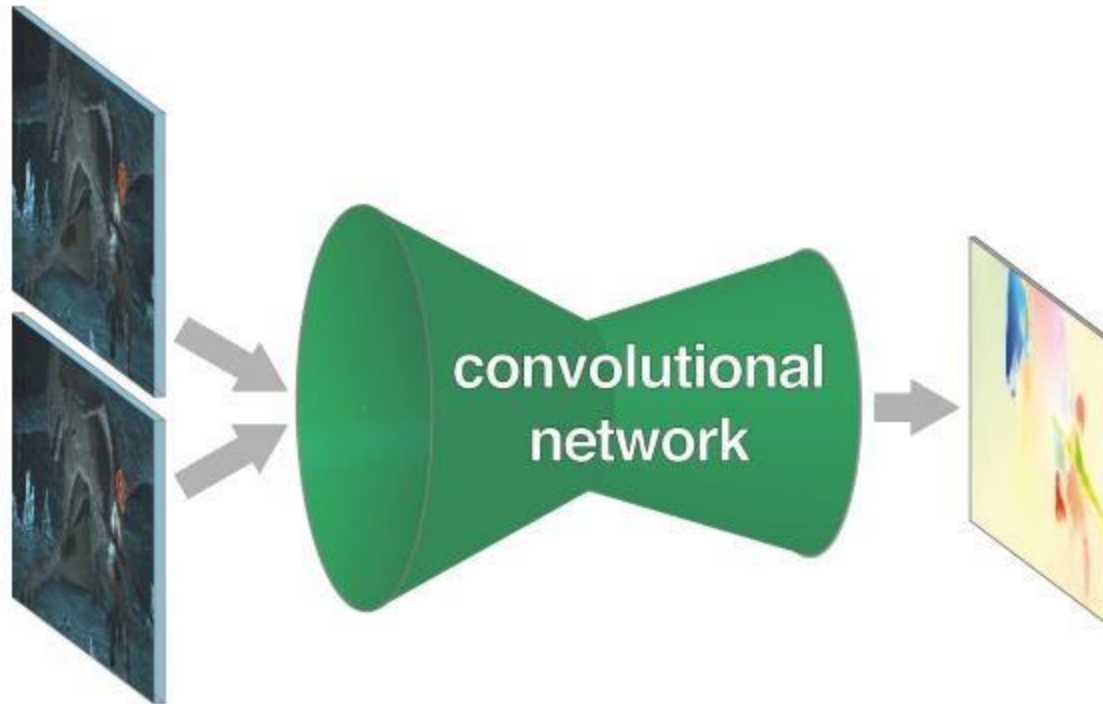
- Start with something similar to Lucas-Kanade
- + gradient constancy
- + energy minimization with smoothing term
- + region matching
- + keypoint matching (long-range)



Region-based +Pixel-based +Keypoint-based

[Large displacement optical flow](#), Brox et al., CVPR 2009

Recent Trends



[DeepFlow: Large displacement optical flow with deep matching](#). ICCV 2013

[FlowNet: Learning Optical Flow with Convolutional Networks](#). ICCV 2015

[Flow fields: Dense correspondence fields for highly accurate large displacement optical flow estimation](#). ICCV 2015

[A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation](#). CVPR 2016

[FlowNet 2.0: Evolution of Optical Flow Estimation with Deep Networks](#). CVPR 2017

[Optical flow estimation using a spatial pyramid network](#). CVPR 2017

[Unsupervised Deep Learning for Optical Flow Estimation](#). AAAI 2017

[Semi-supervised learning for optical flow with generative adversarial networks](#). NIPS 2017

Acknowledgements

- Thanks to the following researchers for making their teaching/research material online
 - Forsyth
 - Steve Seitz
 - Noah Snavely
 - J.B. Huang
 - Derek Hoiem
 - D. Lowe
 - A. Bobick
 - S. Lazebnik
 - K. Grauman
 - R. Zaleski
 - Leibe
 - And many more

Dealing with larger movements:
Iterative refinement

Dealing with larger movements: Iterative refinement

Original (x,y) position



1. Initialize $(x', y') = (x, y)$

$$I_t = I(x', y', t+1) - I(x, y, t)$$

Dealing with larger movements: Iterative refinement

Original (x,y) position



1. Initialize $(x', y') = (x, y)$

$$I_t = I(x', y', t+1) - I(x, y, t)$$

2. Compute (u,v) by

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

2nd moment matrix for feature
patch in first image

displacement

Dealing with larger movements: Iterative refinement

Original (x,y) position



1. Initialize $(x', y') = (x, y)$

2. Compute (u,v) by

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

2nd moment matrix for feature patch in first image

displacement

$$I_t = I(x', y', t+1) - I(x, y, t)$$



Dealing with larger movements: Iterative refinement

Original (x,y) position



1. Initialize $(x', y') = (x, y)$

2. Compute (u, v) by

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

2nd moment matrix for feature patch in first image

displacement

$$I_t = I(x', y', t+1) - I(x, y, t)$$



3. Shift window by (u, v) : $x' = x' + u$; $y' = y' + v$;

Dealing with larger movements: Iterative refinement

Original (x,y) position



1. Initialize $(x', y') = (x, y)$

2. Compute (u, v) by

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

2nd moment matrix for feature patch in first image

displacement

$$I_t = I(x', y', t+1) - I(x, y, t)$$



3. Shift window by (u, v) : $x' = x' + u$; $y' = y' + v$;

4. Recalculate I_t

Dealing with larger movements: Iterative refinement

Original (x,y) position



1. Initialize $(x', y') = (x, y)$

2. Compute (u, v) by

$$\begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = - \begin{bmatrix} \sum I_x I_t \\ \sum I_y I_t \end{bmatrix}$$

2nd moment matrix for feature patch in first image

displacement

$$I_t = I(x', y', t+1) - I(x, y, t)$$



3. Shift window by (u, v) : $x' = x' + u$; $y' = y' + v$;

4. Recalculate I_t

5. Repeat steps 2-4 until small change

- Use interpolation for subpixel values